Financieel

A walkthrough of the codebase

Bram Duthoo

June 12, 2026

Contents

1 Introduction **5**
1.1 What Financieel does . . . 5
1.2 The core idea . . . 5
1.3 How this course is organised . . . 6

2 Foundations **7**
2.1 How a website works . . . 7
2.1.1 Two computers talking . . . 7
2.1.2 Frontend and backend . . . 7
2.1.3 The three roles, and what fills them in our project . . . 8
2.2 The three languages of the browser . . . 8
2.2.1 HTML: the structure . . . 9
2.2.2 CSS: the appearance . . . 9
2.2.3 JavaScript: the behaviour . . . 10
2.3 JavaScript, the working language . . . 10
2.3.1 Declaring variables . . . 10
2.3.2 Functions . . . 11
2.3.3 Objects . . . 11
2.3.4 Arrays . . . 12
2.3.5 Strings with embedded values . . . 12
2.3.6 Truthiness and three shortcut operators . . . 12
2.3.7 Modules . . . 13
2.4 React . . . 13
2.4.1 The problem React solves . . . 13
2.4.2 Components . . . 14
2.4.3 JSX . . . 14
2.4.4 Props: passing data in . . . 15
2.4.5 State: data that changes . . . 15
2.4.6 Re-rendering, and why state is copied rather than edited . . . 16
2.4.7 Effects: doing things besides rendering . . . 16
2.4.8 The pattern of a typical page . . . 17
2.5 Tailwind CSS . . . 18
2.5.1 The idea: utility classes . . . 18
2.5.2 Reading the class names . . . 18
2.5.3 How Tailwind gets into the app . . . 19
2.6 Vite . . . 19

2.6.1 Why a build tool exists . . . 19
2.6.2 Development mode . . . 19
2.6.3 Production builds and environment variables . . . 20
2.7 The backend: databases and Supabase . . . 20
2.7.1 Why the app needs a database . . . 20
2.7.2 What a relational database is . . . 21
2.7.3 How the frontend talks to Supabase . . . 21
2.7.4 Asynchronous code: `async` and `await` . . . 22
2.7.5 Authentication . . . 22
2.7.6 Logic inside the database . . . 23
2.8 From laptop to live website: Git, GitHub and Vercel . . . 23
2.8.1 Git: version control . . . 23
2.8.2 GitHub: the shared copy . . . 23
2.8.3 Vercel: hosting and automatic deployment . . . 24
2.9 The security model . . . 24
2.10 The whole picture . . . 25

3 The Database Schema **27**
3.1 Introduction . . . 27
3.2 Conventions used in every table . . . 27
3.3 Table: wallets . . . 28
3.3.1 What the table represents . . . 28
3.3.2 Schema . . . 28
3.3.3 Columns . . . 29
3.3.4 Foreign key relationships . . . 30
3.3.5 Design notes . . . 30
3.4 Table: transactions . . . 31
3.4.1 What the table represents . . . 31
3.4.2 Schema . . . 31
3.4.3 Columns . . . 31
3.4.4 Foreign key relationships . . . 32
3.4.5 Design notes . . . 32
3.5 Table: recurring_rules . . . 32
3.5.1 What the table represents . . . 32
3.5.2 Schema . . . 33
3.5.3 Columns . . . 33
3.5.4 Foreign key relationships . . . 34
3.5.5 Design notes . . . 34
3.6 Table: income_entries . . . 34
3.6.1 What the table represents . . . 34
3.6.2 Schema . . . 35
3.6.3 Columns . . . 35
3.6.4 Foreign key relationships . . . 35
3.6.5 Design notes . . . 36
3.7 Table: income_recurring . . . 36
3.7.1 What the table represents . . . 36
3.7.2 Schema . . . 36
3.7.3 Columns . . . 36

3.7.4 Foreign key relationships . . . 37
3.8 Table: income_templates . . . 37
3.8.1 What the table represents . . . 37
3.8.2 Schema . . . 37
3.8.3 Columns . . . 37
3.8.4 Foreign key relationships . . . 38
3.8.5 Design notes . . . 38
3.9 Table: income_distribution_rules . . . 38
3.9.1 What the table represents . . . 38
3.9.2 Schema . . . 38
3.9.3 Columns . . . 38
3.9.4 Foreign key relationships . . . 39
3.9.5 Design notes . . . 39
3.10 Table: budget_allocations . . . 39
3.10.1 What the table represents . . . 39
3.10.2 Schema . . . 39
3.10.3 Columns . . . 40
3.10.4 Foreign key relationships . . . 40
3.10.5 Design notes . . . 40
3.11 Table: settings . . . 40
3.11.1 What the table represents . . . 40
3.11.2 Schema . . . 41
3.11.3 Columns . . . 41
3.11.4 Foreign key relationships . . . 41
3.11.5 Design notes . . . 41
3.12 The two balance functions . . . 42
3.12.1 Why they exist . . . 42
3.12.2 Function definitions . . . 42
3.12.3 How they are called from JavaScript . . . 43
3.13 Row Level Security . . . 43
3.13.1 What RLS does . . . 43
3.13.2 Current policies . . . 43
3.13.3 What the current policy enforces . . . 44
3.13.4 The current limitation and the planned tightening . . . 44
3.14 How the tables connect . . . 45

4 The codebase, script by script **46**
4.1 The src folder . . . 46
4.1.1 The lib folder . . . 46
4.1.1.1 supabase.js . . . 46
4.1.1.2 recurringUtils.js . . . 47
4.1.1.3 distributeIncome.js . . . 50
4.1.1.4 dashboardCalcs.js . . . 52
4.1.2 The frame . . . 55
4.1.2.1 main.jsx . . . 55
4.1.2.2 index.css . . . 56
4.1.2.3 App.jsx . . . 57
4.1.2.4 Layout.jsx . . . 59

4.1.2.5 Login.jsx . . . 60
4.1.3 The wallets feature . . . 62
4.1.3.1 pages/Wallets.jsx . . . 63
4.1.3.2 pages/WalletDetail.jsx . . . 64
4.1.3.3 components/WalletCard.jsx . . . 67
4.1.3.4 components/WalletModal.jsx . . . 67
4.1.3.5 components/RecurringRules.jsx . . . 69
4.1.3.6 components/TransactionChecklist.jsx . . . 71
4.1.3.7 components/UpcomingPayments.jsx . . . 72
4.1.3.8 components/PaymentHistory.jsx . . . 73
4.1.3.9 components/VariableTransactionForm.jsx . . . 74
4.1.3.10 components/VariableTransactionList.jsx . . . 76
4.1.3.11 components/WalletTrendsChart.jsx . . . 77
4.1.4 The income feature . . . 78
4.1.4.1 pages/Income.jsx . . . 78
4.1.4.2 pages/IncomeRecurringDetail.jsx . . . 81
4.1.4.3 components/DistributionPopup.jsx . . . 84
4.1.5 The dashboard . . . 86
4.1.5.1 pages/Dashboard.jsx . . . 86
4.1.5.2 components/IncomeSpendingChart.jsx . . . 89
4.1.5.3 components/CashTrendChart.jsx . . . 90
4.1.6 Settings . . . 91
4.1.6.1 pages/Settings.jsx . . . 91
4.2 The project root . . . 94
4.2.1 package.json . . . 94
4.2.2 vite.config.js . . . 96
4.2.3 Tailwind CSS configuration . . . 97
4.2.4 eslint.config.js . . . 97
4.2.5 index.html . . . 98
4.2.6 .gitignore . . . 99

1 Introduction

1.1 What Financieel does

Financieel is a personal finance web application. It tracks where money comes from, where it is supposed to go, and whether reality matches the plan.

Instead of one big bank balance, the app divides money into **wallets**. A wallet is a purpose-bound pot of money. The wallet system exists for three reasons:

1. **Organisation and strategy.** Each wallet carries its own budget and its own rules. Deciding in advance how much of every salary goes to rent, to insurances, to savings, and to free spending turns a vague intention ("I should save more") into a concrete, automated plan.
2. **Tracking against reality.** Every cost is logged in the wallet it belongs to, so the plan is continuously confronted with actual data. A wallet shows not just what was budgeted but what was really spent and what remains.
3. **The total financial picture.** Because every euro is assigned somewhere, the wallets together always add up to a complete overview: how much money there is, where it is sitting, what it is reserved for, and whether upcoming obligations are covered.

1.2 The core idea

Three concepts carry the entire application. Everything in the codebase exists to support one of them.

Wallets are the unit of organisation. Every wallet has a type that determines how it behaves:

Type	Behaviour
Fixed	Driven by recurring payment rules; no manual entries. Rent is the classic example.
Variable	Manual cost entries against a monthly budget. Can be *accumulating* (unused budget carries over, e.g. Holidays) or *capped* (the balance has a maximum, e.g. Clothing).

Type	Behaviour
Investment	Tracks assets and their value over time (planned).
Unallocated	A system wallet that automatically collects money not assigned anywhere else.

Income flows in and is distributed. Income enters through one door (the Income page) and is split across wallets either automatically (recurring income with saved distribution rules) or interactively (a distribution popup for one-off income). Money the user does not assign lands in the Unallocated wallet.

Balances are running totals. Each wallet has a balance: income distribution adds to it, confirmed payments and logged costs subtract from it. The balance is stored, not recalculated. Every change goes through a pair of database functions that increment or decrement it. A balance can go negative; the app never pretends money exists that does not.

1.3 How this course is organised

Chapter 2, Foundations, explains the technologies the app is built with (JavaScript, React, Vite, Supabase, Tailwind, Vercel) and the concepts a reader coming from Python or R has not met yet: components, hooks, async requests, databases, authentication, and the security model.

Chapter 3, The codebase script by script, walks through every file in the project: the goal of the file, the steps its logic performs, and the code that performs them.

The appendix, Local setup, covers cloning the repository, installing dependencies, configuring the environment, and running the app on your own machine.

2 Foundations

This chapter explains how a website works and which building blocks are needed to make one. Every section first explains a concept from the ground up, then shows why this project uses it and how it appears in our code. By the end of the chapter you should be able to look at any file in the project and know what role it plays in the whole.

2.1 How a website works

2.1.1 Two computers talking

Every website involves two computers. Your own computer runs a **browser** (Chrome, Firefox, Safari). Somewhere else in the world a **server** runs day and night, holding the website's files and data. A server is not a special kind of machine; it is an ordinary computer whose job is to wait for requests and answer them.

When you type an address like `financieel-sepia.vercel.app` and press enter, the following happens:

1. The browser sends a **request** across the internet: "please give me this page."
2. The server answers with a **response**: a set of files.
3. The browser reads those files and draws the page on your screen.

This request and response cycle is the heartbeat of the web. It does not happen once per website but constantly: every image, every piece of data, every saved change is its own little request and response. The agreed format these messages follow is called **HTTP**, which is why addresses begin with `https://` (the *s* means the connection is encrypted).

2.1.2 Frontend and backend

The files the server sends are not the whole story. A useful application also needs to *remember* things. In our case: wallets, transactions, income, budgets. This splits every web application into two halves:

- The **frontend** is everything that runs in the browser: the screens, the buttons, the charts, the logic that reacts when you click. The frontend is sent to your browser and executes there, on your machine.

- The **backend** is everything that stays on the server side: most importantly the **database**, the permanent storage where all data lives, plus the rules about who may read or change that data.

The frontend is temporary by nature. Close the browser tab and it is gone. The next time you open the site, a fresh copy is sent over. The backend is permanent: the database keeps its contents whether or not anyone is looking at the site. The frontend therefore continuously asks the backend for data ("give me all wallets") and sends changes back ("save this new transaction").

2.1.3 The three roles, and what fills them in our project

Putting it together, every web application needs three things, and our project fills each role with a specific service:

Role	What it does	In our project
Frontend	The application itself, running in the browser	Built with React, written by us
Backend	Stores data permanently, checks who is allowed to do what	Supabase, a hosted database service
Hosting	A server that hands the frontend files to any visitor	Vercel

Two of the three are services we rent rather than build. Supabase gives us a professional database without managing a server ourselves. Vercel publishes our frontend to the world and keeps it online. The part we actually write, the tens of files in this repository, is almost entirely frontend.

One more tool sits outside this trio: **GitHub**, which stores the history of our code and connects our laptops to Vercel. Section 2.8 covers it. With the map in place, the next sections zoom into each part, starting with the languages the browser itself understands.

2.2 The three languages of the browser

A browser can interpret exactly three languages, and each has one fixed role. There is no choice involved: whoever builds for the web uses these three.

Language	Role	Question it answers
HTML	Structure	What is on the page?
CSS	Appearance	What does it look like?
JavaScript	Behaviour	What happens when the user does something?

2.2.1 HTML: the structure

HTML describes a page as a tree of **elements**. An element is written with tags: an opening tag, content, and a closing tag.

```
<button>Add transaction</button>
```

Elements nest inside each other, which is what creates the tree. A page is one big element (`<html>`) containing a header section (`<head>`, invisible metadata) and a body (`<body>`, the visible content), which in turn contains sections, headings, paragraphs, buttons, input fields and so on. Elements can carry **attributes**, named settings inside the opening tag:

```
<div id="root"></div>
```

Here a `div` (a neutral container element) carries the attribute `id` with value `root`, giving this element a name other code can find it by.

Our project contains exactly one HTML file, `index.html`, and its body is nearly empty: it holds the single `<div id="root">` above and one line that loads our JavaScript. That is no accident. In our project, HTML provides only the empty stage; everything the user actually sees is created by JavaScript while the app runs. Why we build it this way becomes clear in the React section (2.4).

2.2.2 CSS: the appearance

Left alone, HTML renders as black text on a white background. CSS (Cascading Style Sheets) assigns visual properties to elements: colours, sizes, spacing, fonts, position. A CSS rule selects elements and sets properties on them:

```
button {
  background-color: #111827;
  color: white;
  border-radius: 8px;
  padding: 8px 16px;
}
```

This rule says: every `button` element gets a dark background, white text, rounded corners, and some inner spacing.

Two CSS ideas are worth knowing because the entire layout of our app rests on them.

The box model. Every element is a rectangular box with four layers, from inside out: the content itself, **padding** (space between content and border), the **border**, and **margin** (space pushing other elements away). Nearly all visual spacing in any web page is tuning of these layers.

Flexbox. Flexbox is the CSS system for arranging elements next to or below each other in a controlled way: in a row or a column, centred or spread out, fixed width or stretching to fill space. Our app's outer frame, the sidebar next to the page content, is one flexbox row. The rows inside

every table and list are flexboxes too. When you later see classes like `flex`, `items-center` or `justify-between` in our code, they are instructions to this system.

Our project barely contains any handwritten CSS: the file `src/index.css` is one line long. Instead of writing rules like the example above, we use Tailwind, a system that provides thousands of tiny ready-made classes. How that works, and why we chose it, is section 2.5. Tailwind is still CSS underneath; understanding the box model and flexbox is understanding Tailwind.

2.2.3 JavaScript: the behaviour

HTML and CSS are *descriptions*; neither can compute anything or react to the user. JavaScript is the programming language of the browser: a full language with variables, functions, conditions and loops, in spirit much like the languages you already know from data analysis.

JavaScript can do two special things no other code in the browser can:

1. **React to events.** Clicks, typing, key presses. You attach a function to an event and the browser calls it at the right moment.
2. **Modify the page while it is on screen.** When the browser reads HTML it builds an internal tree of objects representing every element, called the **DOM** (Document Object Model). JavaScript can change that tree at any time: add elements, remove them, change their text. The browser instantly redraws to match. The page can therefore change continuously without ever reloading.

These two abilities together make applications possible. When you click "Add transaction" in our app, a JavaScript function runs; it sends the new transaction to the database, then modifies the DOM so the new row appears in your list. No page reload, no new request for HTML; the page simply transforms under your cursor.

In our project, JavaScript is not just a layer on top; it *is* the application. Every file in the `src/` folder is JavaScript (or JSX, a JavaScript extension introduced in section 2.4). The next section covers the parts of the language itself that you need in order to read those files.

2.3 JavaScript, the working language

This section covers the parts of JavaScript you need in order to read the project's code. It assumes you know what variables, functions and loops are in general, and explains everything specific to JavaScript from scratch.

2.3.1 Declaring variables

A variable is created with one of two keywords:

```
const budget = 450
let counter = 0
```

`const` declares a name that cannot be pointed at something else afterwards. `let` declares one that can be reassigned. Our code uses `const` almost everywhere: most values are computed once and then only read, and declaring them `const` makes that guarantee visible. Note that `const` protects the name, not the contents: an object or list declared with `const` can still be modified internally.

2.3.2 Functions

The classic way to define a function:

```
function handleLogin() {
  // ...
}
```

JavaScript also has a compact second form called the **arrow function**, which our code uses constantly:

```
const fmt = (n) => `€${Number(n).toFixed(2)}`
```

Read `=>` as "maps to": `fmt` takes `n` and maps it to a formatted text. When the body is one expression, its value is returned automatically, no `return` keyword needed. With braces, the arrow function behaves like a normal function body and needs an explicit `return`.

Arrow functions matter because in JavaScript, functions are values: they can be stored in variables, put inside objects, and most importantly **passed into other functions**. A function passed into another function, to be called at the right moment, is called a **callback**. The pattern is everywhere in web code: "when the user clicks, call this function", "for every element of this list, call this function". Arrow functions are the convenient way to write callbacks in place:

```
// from Wallets.jsx - when this button is clicked, run openCreate
<button onClick={() => openCreate()}>
```

2.3.3 Objects

An object is a collection of named values, written with braces:

```
const wallet = { name: 'Rent', budget: 450, type: 'fixed' }
wallet.budget          // read a field: 450
wallet.budget = 500    // change a field
```

Objects are the universal data container of JavaScript. A wallet, a transaction, a form's current contents, the response from the database: in our code, each is an object. Three shorthand notations appear often:

Object shorthand. When the field name and the variable name are identical, write it once. `{ name, amount }` means `{ name: name, amount: amount }`.

Destructuring. Unpacking fields of an object into variables in one statement:

```
const { data, error } = await supabase.from('wallets').select('*')
```

The right-hand side returns one object with fields `data` and `error`; the statement pulls both into their own variables. Every database query in the project uses exactly this shape.

The spread operator `...`. Copies all fields of an object into a new object, after which individual fields can be overridden:

```
// from RecurringRules.jsx - change one field of a form object
setForm(f => ({ ...f, [key]: val }))
```

This means: build a fresh object, copy everything from `f` into it, then set one field to a new value. The original is untouched. The bracket notation `[key]` makes the field name itself come from a variable. Why our code copies objects instead of just changing them in place is a React rule explained in section 2.4.6.

2.3.4 Arrays

An array is an ordered list, written with square brackets. The interesting part is how lists are processed. JavaScript work on lists is done by calling **methods on the array**, each taking a callback:

```
wallets.map(w => w.name)            // transform every element → list of names
wallets.filter(w => w.is_active)    // keep only elements passing a test
wallets.find(w => w.id === id)      // the first element passing a test
```

The fourth essential method is `reduce`, which boils a list down to one value. It carries an accumulator from element to element:

```
// from Dashboard.jsx - total income this month
const totalIncome = income.reduce((s, e) => s + Number(e.amount), 0)
```

`reduce` starts the accumulator `s` at `0` and, for every entry `e`, replaces `s` with `s + Number(e.amount)`. The final accumulator is the total. Reading `map`, `filter`, `find` and `reduce` fluently is the single most useful skill for reading this codebase: nearly all data handling is chains of these four.

2.3.5 Strings with embedded values

Text in backticks may embed expressions with `${}`:

```
note: `Income distribution - ${sourceName}`
```

The expression inside `${}` is evaluated and spliced into the text.

2.3.6 Truthiness and three shortcut operators

In a condition, JavaScript treats certain values as false: `false`, `0`, `""` (empty text), `null`, `undefined` and `NaN`. Everything else counts as true. Three operators build on this and appear on nearly every

page of the codebase.

`&&` (and). `a && b` evaluates to `b` only if `a` is truthy; otherwise it stops and evaluates to `a`. Besides its normal logical use, the interface code uses it to show something only under a condition, as section 2.4.3 will show.

`??` (fallback). `a ?? b` gives `b` only when `a` is `null` or `undefined`. Our queries use it to fall back to an empty list when no data came back:

```
setWallets(data ?? [])
```

`?.` (safe access). `obj?.field` gives `undefined` instead of crashing when `obj` is missing. On functions, `fn?.()` calls `fn` only if it exists:

```
onRulesChanged?.()   // notify the parent, but only if a callback was provided
```

2.3.7 Modules

Every file in the project is a **module**: a unit with its own private scope that explicitly shares and consumes values. Sharing is done with `export`, consuming with `import`:

```
// lib/supabase.js - share one value with the whole app
export const supabase = createClient(supabaseUrl, supabaseAnonKey)

// any other file - use it
import { supabase } from '../lib/supabase'
```

A module may additionally declare one **default export**, imported without braces. Every page and component in our project is a default export:

```
export default function Dashboard() { ... }    // pages/Dashboard.jsx
import Dashboard from './pages/Dashboard'       // App.jsx
```

The import path starting with `./` or `../` is a file path relative to the importing file. An import path without those, like `import { useState } from 'react'`, refers to an installed package (section 2.6 explains where packages live).

2.4 React

2.4.1 The problem React solves

With plain JavaScript you can already build an interactive page: listen for a click, compute something, modify the DOM. For a small page that is fine. For an application like ours it collapses under its own weight, for one central reason: **keeping the screen synchronised with the data becomes unmanageable.**

Consider what happens in our app when one transaction is added. The transaction list must show a new row. The wallet balance in the header must change. The spending bar must grow. The

dashboard's totals, charts and alerts must all update. With plain JavaScript, the programmer must remember every spot on the screen that depends on the changed data and write code to update each one by hand. Forget one and the screen silently shows stale numbers. As an app grows, these dependencies multiply until no one can track them.

React inverts the model. Instead of writing *how to update* the screen, you write *what the screen should look like* for any given data. When data changes, React re-evaluates that description and works out by itself which parts of the DOM must change. The programmer never touches the DOM directly. The screen becomes a pure consequence of the data, and stale displays become impossible by construction.

2.4.2 Components

A React application is built from **components**. A component is a JavaScript function that returns a description of a piece of interface:

```
export default function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>
      ...
    </div>
  )
}
```

Components nest. Our `App` component contains a `Layout` component, which contains the sidebar and whichever page is active; the `Wallets` page contains one `WalletCard` component per wallet. The whole interface is one big tree of components, and that tree is the architecture of the frontend. Building a screen means composing it from components; building a feature usually means writing a new component or extending one.

Components are reusable by design: `WalletCard` is written once and rendered many times, once per wallet, each time with different data. How data is fed in is the subject of props (2.4.4).

2.4.3 JSX

The HTML-like syntax inside those functions is called **JSX**. It is not HTML; it is a JavaScript extension that lets you write the structure of the interface directly inside code. The build tool (section 2.6) translates it into ordinary JavaScript before the browser sees it.

JSX behaves like a templating language fused into the code. Three rules carry almost everything:

Braces embed JavaScript. Anything inside `{}` is evaluated as a JavaScript expression and its result is rendered:

```
<p>Balance: €{Number(wallet.balance).toFixed(2)}</p>
```

Conditional rendering. Combining braces with the `&&` operator from 2.3.6 shows an element only when a condition holds. The pattern reads "condition and element":

```
// from Login.jsx - the error box exists only when there is an error
{error && (
  <div className="bg-red-50 text-red-600 ...">{error}</div>
)}
```

Lists by mapping. A list of data becomes a list of elements with `map`. Each element must carry a `key` attribute, a stable identifier React uses to track which item is which between updates:

```
// from Wallets.jsx - one card per wallet
{list.map(w => (
  <WalletCard key={w.id} wallet={w} ... />
))}
```

Two cosmetic differences from HTML matter: the attribute for CSS classes is `className` rather than `class` (because `class` is a reserved word in JavaScript), and a component with nothing inside may be written self-closing, like `<WalletCard ... />`.

2.4.4 Props: passing data in

A component receives data through **props** (properties), which look like attributes at the place of use and arrive as one object in the function:

```
// the parent renders:
<WalletCard wallet={w} onEdit={openEdit} onDelete={setDeleteTarget} />

// the component receives:
export default function WalletCard({ wallet, onEdit, onDelete }) { ... }
```

Props flow strictly downward, from parent to child. Notice that two of these props are functions. Passing callbacks down is how children talk back upward: `WalletCard` cannot reach into its parent, but when its edit button is clicked it calls `onEdit(wallet)`, and the parent decides what that means. Data flows down as values; events flow up as function calls. That sentence describes the traffic pattern of the entire frontend.

2.4.5 State: data that changes

Props are given from outside. **State** is data a component owns itself and is allowed to change: the text currently typed in a form field, whether a popup is open, the list of wallets fetched from the database. State is declared with `useState`:

```
// from Login.jsx
const [email, setEmail] = useState('')
```

`useState('')` creates a piece of state with starting value `''` and hands back two things: the current value (`email`) and a function to change it (`setEmail`). The crucial rule: **state is only ever changed through the setter.** Calling `setEmail('x')` does two things: it stores the new value, and it tells React that this component's data changed, so its description of the interface must be re-evaluated. Assigning `email = 'x'` directly would change a local variable and nothing else; React would never know, and the screen would not update.

Functions like `useState` are called **hooks**, recognisable by the `use` prefix. They are React's mechanism for giving plain component functions capabilities like memory and side effects.

2.4.6 Re-rendering, and why state is copied rather than edited

When state changes, React calls the component function again from top to bottom, producing a fresh description of the interface, and compares it with the previous one. Only the differences are applied to the real DOM. This cycle is called a **re-render**, and it is cheap by design: computing descriptions and diffing them is fast, touching the real DOM is minimised.

This model explains a rule you will see throughout the code: state holding an object or list is never edited in place, but replaced with a modified copy:

```
setForm(f => ({ ...f, [key]: val }))     // copy the form, override one field
```

React decides whether state changed by checking whether it received a *different* object. Editing the existing object in place leaves the identity unchanged, and React concludes nothing happened. Building a copy with the spread operator gives React a new object to see. That is the entire reason for the copy pattern from section 2.3.3.

2.4.7 Effects: doing things besides rendering

Rendering must stay pure: a component function computes a description and nothing else. But real components need to *do* things, above all fetch data from the database. Such actions are called **side effects** and have their own hook, `useEffect`:

```
// from Wallets.jsx - load wallets when the page appears
useEffect(() => { fetchWallets() }, [])
```

`useEffect` takes a function to run and a **dependency list** controlling when to run it. The empty list `[]` means: run once, when the component first appears on screen. This is the standard pattern for initial data loading, and nearly every page in the project starts with it. A dependency list with values in it, like `[id]` in `WalletDetail.jsx`, means: run again whenever `id` changes, so the page reloads its data when you navigate from one wallet to another.

An effect may return a cleanup function, which React calls when the component leaves the screen.

`App.jsx` uses this to unsubscribe from login-state notifications so no listener is left running:

```
useEffect(() => {
  const { data: { subscription } } = supabase.auth.onAuthStateChange(...)
  return () => subscription.unsubscribe()
}, [])
```

2.4.8 The pattern of a typical page

Almost every page in the project is built from the same skeleton, which you will now recognise piece by piece:

```
export default function Wallets() {
  const [wallets, setWallets] = useState([])          // 1. state
  const [loading, setLoading] = useState(true)

  useEffect(() => { fetchWallets() }, [])              // 2. load on appear

  async function fetchWallets() {                      // 3. fetch + store
    const { data } = await supabase.from('wallets').select('*')
    setWallets(data ?? [])
    setLoading(false)
  }

  return (                                             // 4. describe the UI
    <div>
      {loading
        ? <p>Loading...</p>
        : wallets.map(w => <WalletCard key={w.id} wallet={w} ... />)}
    </div>
  )
}
```

State at the top, an effect that triggers the first data load, fetch functions that end by storing results in state, and a JSX description that renders whatever the state currently holds. Changing state triggers a re-render, the description is re-evaluated, the screen follows. Once this loop is familiar, most files in `src/pages/` read themselves. The `async` and `await` keywords in the fetch function belong to asynchronous programming, covered with the database itself in section 2.7.

2.5 Tailwind CSS

2.5.1 The idea: utility classes

Section 2.2.2 showed classic CSS: rules in a separate file selecting elements and setting their properties. That approach has a scaling problem similar to the one React solves for behaviour. As an app grows, the style file fills with hundreds of rules, rules begin to overlap and override each other, and changing one safely requires knowing every place it applies.

Tailwind takes a different approach. It provides thousands of tiny ready-made classes, each setting exactly one property, and you style an element by listing the classes it needs directly on the element:

```
<button className="bg-gray-900 text-white px-4 py-2 rounded-lg text-sm font-medium">
  Add Income
</button>
```

Reading left to right: dark background, white text, horizontal padding, vertical padding, rounded corners, small text, medium font weight. These are called **utility classes**. There is no separate style file to maintain and no naming of rules; the element's appearance is fully visible at the element itself. When you delete a component, its styling disappears with it, with nothing left behind to go stale.

2.5.2 Reading the class names

Tailwind names follow a compact system. A short prefix names the property; a suffix names the value on a fixed scale.

Class	Meaning
`p-5`	padding on all sides, step 5 of the spacing scale (1.25rem = 20px)
`px-4`, `py-2`	horizontal and vertical padding
`m-4`, `mb-4`	margin; margin-bottom only
`text-sm`, `text-3xl`	font size
`font-medium`	font weight 500
`bg-white`, `bg-gray-900`	background colour (colour name plus shade 50–950)
`text-gray-600`	text colour
`border`, `border-stone-200`	a border; its colour
`rounded-lg`, `rounded-2xl`	corner rounding, increasing
`flex`, `flex-col`	flexbox container; stack children vertically
`items-center`, `justify-between`	flexbox alignment (cross axis; main axis)
`gap-3`, `space-y-4`	spacing between flex children; vertical gaps between siblings
`w-44`, `h-screen`, `flex-1`	fixed width; full viewport height; absorb remaining space

Class	Meaning
`hover:bg-stone-100`	apply a class only while the mouse is over the element

The bracket notation `text-[11px]` or `bg-[#D85A30]` escapes the fixed scale and uses an exact value; the project's design system uses this for its precise colours.

With this table, the layout code from section 2.2.2 reads as plain language:

```
<div className="flex h-screen bg-gray-50">      // row, full height, light grey page
  <aside className="w-56 bg-white flex flex-col">  // fixed-width white column
  <main className="flex-1 overflow-auto">          // the rest, scrollable
```

2.5.3 How Tailwind gets into the app

Our `src/index.css` contains one line, `@import "tailwindcss"`, and the build tool runs a Tailwind plugin (visible in `vite.config.js`). At build time, Tailwind scans every file in the project for class names and generates a CSS file containing only the classes actually used. Nothing needs to be written or maintained by hand; using a class in JSX is all it takes for it to exist.

2.6 Vite

2.6.1 Why a build tool exists

Two facts about our code make it impossible to hand it to a browser directly. First, JSX is not real JavaScript; something must translate it. Second, the project depends on external packages (React itself, the Supabase client, the icon library), which live outside our source files. A **build tool** solves both: it translates what browsers cannot read and bundles our files and the packages into plain files browsers can. Our build tool is **Vite**.

Before Vite can do anything, the packages must exist on disk. That is the job of **npm**, the package manager that comes with **Node.js** (a program that runs JavaScript outside a browser; Vite itself is JavaScript and runs on it). The file `package.json` at the project root lists every package the project depends on, with version ranges. Running `npm install` reads this list and downloads everything into the `node_modules` folder. This is why a freshly cloned copy of the repository does not run until `npm install` has been executed once, and why `node_modules` is never committed to Git: it is bulky and entirely reproducible from `package.json`.

2.6.2 Development mode

While building the app we run:

```
npm run dev
```

This starts Vite as a local development server, reachable only from your own machine at the address `http://localhost:5173`. It serves the app, translating JSX on the fly, and it watches every source file. When you save a file, Vite pushes the change into the open browser within a fraction of a second, usually without even reloading the page. This tight save-and-see loop is what makes frontend development workable.

`localhost` is a name every computer gives to itself, and `5173` is a **port**, one of many numbered doors a machine can listen on. Nothing about this server is public; a colleague cannot open your localhost.

2.6.3 Production builds and environment variables

For the public site, Vite instead performs a **build** (`npm run build`): it translates and bundles everything into a small set of optimised plain files in a `dist` folder. Those files are what the hosting service actually serves to visitors. We never run this command ourselves; the hosting service does, as section 2.8 describes.

One more Vite responsibility matters for understanding the code: **environment variables**. The file `.env.local` at the project root holds values that must stay out of the codebase, in our case the address of our database and the key used to talk to it. Vite makes any variable whose name starts with `VITE_` available to the code as `import.meta.env.VITE_...`:

```
// lib/supabase.js
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY
```

`.env.local` is deliberately excluded from version control, so these values never appear on GitHub. Section 2.9 explains why this is safe and what these keys can and cannot do.

2.7 The backend: databases and Supabase

2.7.1 Why the app needs a database

Everything described so far lives in the browser and is therefore temporary. State vanishes when the tab closes. For an app whose entire purpose is remembering financial history, the data must live somewhere permanent, reachable from any browser, guarded against unauthorised access. That place is a **database** running on a server: the backend.

Our backend is **Supabase**, a hosted service that bundles three things we would otherwise have to build and operate ourselves: a professional database, user authentication (accounts, passwords, login sessions), and an automatically generated web interface to the database, so the frontend can talk to it directly over the internet.

2.7.2 What a relational database is

Supabase's core is **PostgreSQL**, a relational database. A relational database stores data in **tables**. A table has named, typed **columns** and holds the data as **rows**. Our `wallets` table, simplified:

id	name	type	budget	balance
7f3a…	Rent	fixed	450.00	450.00
91c2…	Holidays	variable	200.00	350.00

Each row is one wallet. The `id` column holds a generated unique identifier, and every table has one. Tables refer to each other through these ids: each row of the `transactions` table carries a `wallet_id` column naming the wallet it belongs to. Such a reference is called a **foreign key**, and it is the "relational" in relational database. The full set of tables, columns and references is called the **schema**; ours has eight tables (wallets, transactions, income entries, recurring rules, income recurring, income templates, distribution rules, budget allocations, settings) and is documented table by table in ARCHITECTURE.md.

The language for talking to a relational database is **SQL**. We used it to create the schema, and a few pieces of project logic live in the database as SQL functions. Reading the codebase requires almost no SQL, but the shape of a query is worth recognising:

```sql
select * from wallets where is_active = true;
```

2.7.3 How the frontend talks to Supabase

The frontend cannot open the database directly; it sends HTTP requests (the same request and response mechanism from section 2.1) to Supabase's web interface, called an **API** (Application Programming Interface): an entry point for programs rather than people. Rather than constructing these requests by hand, we use the **supabase-js** client library, which wraps them in readable JavaScript. The single connection object is created once, in `lib/supabase.js`, and imported everywhere:

```
import { createClient } from '@supabase/supabase-js'
export const supabase = createClient(supabaseUrl, supabaseAnonKey)
```

Queries then read almost like sentences:

```
const { data, error } = await supabase
  .from('transactions')
  .select('*')
  .eq('wallet_id', walletId)
  .order('date', { ascending: false })
```

This fetches all columns of the transactions whose `wallet_id` matches, newest first. Writing follows the same style with `.insert({...})`, `.update({...})` and `.delete()`, each combined with filters

like `.eq(...)`. Every response is an object with `data` (the result, or `null`) and `error` (the problem, or `null`), which is why the destructuring pattern from 2.3.3 opens nearly every query in the project.

2.7.4 Asynchronous code: `async` and `await`

A database query crosses the internet and takes time, perhaps fifty milliseconds, perhaps two seconds. JavaScript does not stop the world while waiting; the operation starts, and the program continues. Code that starts something and finishes later is **asynchronous**.

JavaScript marks the result of such an operation with an object called a Promise, but day to day you only need the two keywords built on top of it. A function declared `async` may contain waiting points; inside it, `await` pauses *that function* until a result arrives, then hands the result over:

```
async function fetchWallets() {
  const { data } = await supabase.from('wallets').select('*')
  setWallets(data ?? [])
}
```

Without `await`, `data` would be read before the response existed. While one function is paused at an `await`, the rest of the app keeps running: the interface stays responsive, clicks still work. This is why every fetch function in the project is `async`, and why loading states exist: between starting a query and its `await` completing, the page shows "Loading…".

When several independent queries are needed at once, the project starts them together and waits for all of them, rather than awaiting one after another:

```
// from Dashboard.jsx - two queries, in parallel
const [{ data: w }, { data: i }] = await Promise.all([
  supabase.from('wallets').select('*'),
  supabase.from('income_entries').select('*'),
])
```

2.7.5 Authentication

The app must know who is using it before showing anything. Supabase provides this: it stores user accounts and verifies passwords, and the client library exposes the login operations. The login page calls:

```
const { error } = await supabase.auth.signInWithPassword({ email, password })
```

On success, Supabase issues a **session**: a signed token stored by the browser and attached automatically to every subsequent database request, proving on each request who is asking. The frontend reacts to login state through two calls in `App.jsx`: `supabase.auth.getSession()` reads the current session when the app starts, and `supabase.auth.onAuthStateChange(...)` notifies the app the moment the user logs in or out. `App.jsx` keeps the session in state and uses it as a switch: no session renders the login page, a session renders the application. Logging out (`supabase.auth.signOut()`)

clears the session, the listener fires, the state empties, and the same switch sends the user back to the login screen.

2.7.6 Logic inside the database

One last backend concept appears in the code. Wallet balances must change atomically: read and write as one indivisible step, so two simultaneous changes cannot overwrite each other. For this, two small functions live inside the database itself, written in SQL: `increment_wallet_balance` and `decrement_wallet_balance`. The frontend invokes them by name through a **remote procedure call**:

```
await supabase.rpc('decrement_wallet_balance', {
  p_wallet_id: walletId,
  p_amount: item.rule.amount,
})
```

Whenever you see `supabase.rpc(...)` in the code, a function stored in the database is doing the work, not JavaScript.

2.8 From laptop to live website: Git, GitHub and Vercel

2.8.1 Git: version control

Code is written in small steps, and mistakes happen. **Git** is a program that records the history of a project as a chain of snapshots, so any earlier state can be inspected or restored. It runs locally on your machine, inside the project folder.

Three commands carry the daily routine. After changing some files:

```
git add .
git commit -m "Add transaction detail modal"
git push
```

`git add .` stages the changes (selects what the snapshot will contain). `git commit` takes the snapshot, with a message describing it. Each commit records who, when, what changed, and gets a unique identifier. The history of this project is the chain of all such commits, from "Initial React + Vite setup" to today, and any of them can be revisited.

`git push` is where GitHub enters.

2.8.2 GitHub: the shared copy

GitHub is a website that hosts Git projects (called **repositories**). Our repository lives at github.com/bramduthoo/financieel. It serves two purposes: it is an off-machine backup of the entire history, and it is the meeting point through which several people work on the same code. `git push` uploads your new commits to GitHub; `git pull` downloads commits others have pushed. Each collaborator works on a full local copy and synchronises through GitHub.

Parallel work is organised with **branches**. A branch is an independent line of development: you split off from the main line, build a feature in isolation over any number of commits, and when it is finished and tested, merge it back. The main line, the branch called `main`, always holds the version of the app that is live. Merging into `main` happens through a **pull request** on GitHub: a page showing every change the branch would introduce, where the other person can review and approve before the merge button is pressed. Nothing reaches `main`, and therefore nothing reaches the live site, without passing this gate.

One rule from section 2.6 bears repeating here: `node_modules` and `.env.local` are listed in a file called `.gitignore` and therefore never enter the repository. The first is bulky and reproducible; the second contains the database keys and must not be published.

2.8.3 Vercel: hosting and automatic deployment

Vercel is the hosting service: the server that hands the app to any visitor of financieel-sepia.vercel.app. It is connected to the GitHub repository and watches the `main` branch. Every time new commits arrive on `main`, Vercel automatically pulls the code, runs the production build from section 2.6.3 (`npm run build`), and puts the resulting files live. The whole pipeline is:

```
edit code → commit → push to GitHub → merge to main → Vercel builds → live
```

No manual uploading exists anywhere in the process. Vercel also stores its own copy of the environment variables (the Supabase address and key), entered once in its dashboard, since it cannot read the `.env.local` file that never left your machine.

2.9 The security model

The app holds financial data, so it is worth being precise about what protects it. Security here is layered: each layer assumes the one before it might fail.

Layer 1: secrets stay out of the repository. The Supabase address and key live in `.env.local`, which Git ignores. Browsing the GitHub repository reveals no credentials.

Layer 2: the key in the browser is the harmless one. Supabase issues two keys. The **anon key** is the one our frontend uses, and it is designed to be visible (anyone can find it in their browser's network traffic). It grants no data access by itself; it only identifies which Supabase project a request is for. The **service role key**, which bypasses all protection, is never used in this project and exists only in the Supabase dashboard.

Layer 3: authentication. Reading or writing any table requires a valid session, obtained only by logging in with the correct email and password (section 2.7.5). Requests without a session are rejected.

Layer 4: the database enforces this itself. The rule "only authenticated users may access these tables" is not frontend code; it is configured inside the database as **Row Level Security** (RLS). Every table carries a policy checking that the request comes from an authenticated session. This is

the decisive layer: even someone who takes the anon key and crafts their own requests, bypassing our app entirely, hits the same wall, because the check happens in the database, after the request arrives, on every single row.

A current limitation is worth naming: the policies check *that* someone is logged in, not *who*. With one user account this is equivalent. The moment a second person gets an account, they would see the same data. True multi-user support (giving every row an owner and tightening the policies to "each user sees only their own rows") is the planned next step, and it strengthens exactly this layer.

2.10 The whole picture

To close the chapter, here is one complete action traced through every layer just described. The user opens a fixed wallet and ticks off the pending rent payment of €450.

1. The page renders from state. `WalletDetail.jsx` is on screen. When it appeared, a `useEffect` ran its fetch functions; `await`ed Supabase queries returned the wallet, its recurring rules and its transactions; the results landed in state via setters; React rendered the page from that state. The pending checklist visible on screen is the component `TransactionChecklist` mapping over computed pending items.

2. A click becomes a function call. The circle next to "Rent" carries an `onClick` callback. The click opens the confirmation modal (a state change: `setConfirmItem(item)` triggers a re-render which now includes the modal). The user confirms.

3. The frontend writes to the backend. The confirm handler is an `async` function. It sends an insert to Supabase, recording the payment as a confirmed transaction:

```
await supabase.from('transactions').insert({
  wallet_id: walletId,
  recurring_rule_id: item.rule.id,
  amount: item.rule.amount,
  type: 'debit',
  date: item.dateStr,
  is_confirmed: true,
  completed_at: now,
})
```

The supabase-js client turns this into an HTTP request and attaches the session token. At Supabase, Row Level Security checks the session before the row is written.

4. Database logic adjusts the balance. The handler then calls the stored function, and the database subtracts €450 from the wallet's balance atomically:

```
await supabase.rpc('decrement_wallet_balance', {
  p_wallet_id: walletId, p_amount: item.rule.amount,
})
```

5. The screen catches up by re-fetching. The handler ends by calling the fetch functions again and notifying the parent page (`onBalanceChanged?.()`). Fresh data arrives, setters store it, React re-renders: the item leaves the pending list, the balance pill in the header shows €450 less, and the History tab will list the payment. Nothing on the screen was edited directly; the data changed, and the screen followed.

Every concept of this chapter appears in those five steps: components and props, state and effects, async requests, the client library, RLS, and database functions. Chapter 3 now walks through the codebase file by file, and each file will be a variation on machinery you have already seen.

3 The Database Schema

3.1 Introduction

Chapter 2.7 introduced the database as the permanent layer of the application: the place that holds all data across sessions, across devices, and across restarts. Section 2.7.2 described what a relational database is, how it organises data into tables with typed columns, and how tables reference each other through foreign keys. This chapter documents the schema itself: the precise, permanent shape of the data the application works with.

The schema comprises nine tables. Three of them deal primarily with wallets and the money flowing through them: `wallets` defines every wallet and stores its running balance; `transactions` records every individual credit and debit; and `budget_allocations` tracks a wallet's budget history across time. Four tables belong to the income system: `income_entries` logs each income event, `income_recurring` defines recurring salary or payment schedules, `income_templates` stores named amount presets, and `income_distribution_rules` encodes how each recurring income should be split across wallets on arrival. One table, `recurring_rules`, holds the payment schedule definitions that drive fixed wallets. One final table, `settings`, holds the single row of application-wide configuration.

Understanding the schema is understanding the backbone of the application. Every piece of logic in the frontend either reads from or writes to this structure, and the constraints, defaults, and relationships defined here enforce rules that JavaScript code relies on but cannot enforce on its own.

3.2 Conventions used in every table

Every table in the schema shares a small set of structural conventions. Learning them once means not having to re-explain them nine times.

The primary key. Every table has a column named `id` of type `uuid`. A UUID is a 128-bit random identifier, written as a hexadecimal string in groups (for example, `7f3a1c8e-4d2b-4e1a-9a3c-1f2e3d4c5b6a`). The column carries the constraint `primary key`, which means every row's id is unique across the table and no row may have a null id. The default is `gen_random_uuid()`, a built-in PostgreSQL function that generates a new UUID at the moment a row is inserted. The frontend therefore never

needs to supply an id when creating a record; the database generates one automatically.

The reason for using UUIDs rather than sequential integers (1, 2, 3, …) is safety: a sequential id leaks how many records exist and makes it easy to guess neighbouring ids. A random UUID reveals nothing about the rows around it.

The creation timestamp. Every table has a column `created_at` of type `timestamptz`, meaning a timestamp with time-zone information included. Its default is `now()`, so it is filled automatically when a row is inserted. The application uses it for sorting history tables and for understanding when records were created.

The absence of updated_at. Almost no table in this schema carries an `updated_at` column. The reason is that very few records are ever edited in place. Wallets are occasionally renamed. Settings are occasionally changed. But transactions are never edited, income entries are never edited, and recurring rules are never edited: if a rule changes, a new rule is created and the old one is archived. An `updated_at` column on tables whose rows are written once and never touched again would be noise.

Foreign keys. When one table references another, the referencing column is named `<table>_id`, where `<table>` is the singular name of the referenced table. The column carries a `references` clause pointing to the other table's `id`. Some foreign keys also carry `on delete cascade`, which means: if the referenced row is deleted, the rows that point to it are automatically deleted too. Which keys carry this clause and why is noted in each table's section below.

Deletion by end date. Two tables in the schema, `recurring_rules` and `income_recurring`, hold records that must never be truly deleted, because past transactions or income entries already reference them and their history must remain intact. Instead of deleting a rule when it is retired, the application sets a date in its `end_date` column. A null `end_date` means the rule is currently active. A non-null `end_date` means the rule was retired as of that date. Any query for active rules filters on `end_date is null`.

3.3 Table: wallets

3.3.1 What the table represents

Each row of the `wallets` table represents one named pot of money with its own purpose, budget, and running balance. Wallets are the central organising concept of the application. Section 1.1 explains the four wallet types (fixed, variable, investment, and unallocated) and why dividing money into purpose-bound pots serves a budgeting goal; this section documents the columns that make those concepts concrete in the database.

3.3.2 Schema

```
create table wallets (
  id                    uuid          primary key default gen_random_uuid(),
```

```
  name                  text          not null,
  type                  text          not null,
  budget_type           text          not null,
  budget                numeric(10,2) not null default 0,
  balance               numeric(10,2) not null default 0,
  colour                text,
  icon                  text,
  is_active             boolean       not null default true,
  is_system             boolean       not null default false,
  sort_order            integer       not null default 0,
  cap_reduction_enabled boolean       not null default false,
  cap_reduction_rate    numeric(5,2)  not null default 0,
  created_at            timestamptz   not null default now()
);
```

3.3.3 Columns

type stores one of four values: `fixed`, `variable`, `investment`, or `unallocated`. This single column determines the entire behaviour of the wallet across the interface, from which tabs are shown in `WalletDetail.jsx` to whether the transaction form accepts manual entries. It is the dispatch column for wallet behaviour throughout the application.

budget_type refines the meaning of the `budget` column. Where `type` describes the wallet's role, `budget_type` describes how the budget figure is interpreted:

- `fixed-recurring`: the budget column holds the sum of all recurring payment rules. It is maintained by the application whenever rules change, not edited directly by the user.
- `accumulating`: a variable wallet whose unused monthly budget rolls over, so the balance grows over time until it is spent. A holiday fund is the canonical example.
- `capped`: a variable wallet whose balance is not allowed to exceed the budget amount. Any income distribution that would push the balance above the cap is redirected elsewhere. A clothing budget is a typical example.
- `none`: a wallet with no monthly budget, intended for investment wallets.
- `unallocated`: reserved for the single system wallet that collects any income not assigned elsewhere.

budget holds the monetary value associated with the budget type. For a capped wallet it is the ceiling the balance may not exceed; for an accumulating wallet it is the monthly addition target; for a fixed-recurring wallet it is the sum of the payment rules. The type `numeric(10,2)` means a decimal number with up to ten digits in total and exactly two digits after the decimal point, which matches standard currency precision.

balance is the stored running total: every credit increments it and every debit decrements it. This is a design choice worth understanding. An alternative would be to compute the balance on demand

by summing all transactions for the wallet. The stored approach is faster (one read rather than summing potentially thousands of rows) but requires discipline: the balance is only ever changed through the two dedicated database functions described in section 3.12. Direct updates to this column are never made from the frontend.

colour and **icon** are display-only strings used by the interface to style each wallet card. `colour` holds a hex code such as `#D85A30`; `icon` holds the name of a `lucide-react` icon. Neither has any significance for financial logic.

is_active allows a wallet to be retired without deletion. An inactive wallet no longer appears in the distribution popup or the main wallet list, but its historical transactions remain intact and queryable.

is_system is set to true on exactly one row: the Unallocated wallet. The interface uses this flag to prevent the user from deleting or editing that wallet, since it plays a structural role in the income distribution system. Any income that is not explicitly assigned to another wallet during distribution lands here automatically.

sort_order is an integer that lets the user control the sequence in which wallets are presented. The application writes it when the user reorders their wallet list.

cap_reduction_enabled and **cap_reduction_rate** apply only to capped wallets that have reached their ceiling. When `cap_reduction_enabled` is true and the wallet's balance is already at or above the budget, automated income distributions do not drop to zero: they continue at a reduced rate. `cap_reduction_rate` is a fraction between 0 and 1 stored as `numeric(5,2)`; a value of `0.50` means the wallet continues receiving 50% of its normal allocation while at capacity, with the remaining 50% redirected to the Unallocated wallet. The full logic lives in `distributeIncome.js`. For manual and template income, this reduction is never applied regardless of the cap state, because the user is making an explicit allocation decision in real time.

3.3.4 Foreign key relationships

No other table is referenced by `wallets`. Several tables reference it: `transactions`, `recurring_rules`, `income_distribution_rules`, and `budget_allocations` each carry a `wallet_id` foreign key pointing here. `wallets` is the hub of the dependency graph (see section 3.14).

3.3.5 Design notes

The combination of `type` and `budget_type` might initially appear redundant, but each answers a different question. `type` answers "what kind of wallet is this?" and drives structural decisions: which page components appear, which operations are permitted. `budget_type` answers "how does the budget number behave?" and drives calculation decisions: how the monthly budget is projected, what happens at cap. A fixed wallet is always `fixed-recurring`; variable wallets may be `accumulating` or `capped`. These two orthogonal concepts need two columns.

3.4 Table: transactions

3.4.1 What the table represents

Each row of the `transactions` table records one financial event affecting a wallet: either a credit (money entering the wallet) or a debit (money leaving it). Every wallet balance change is the result of a transaction row. The table is therefore the complete ledger of the application.

3.4.2 Schema

```
create table transactions (
  id                uuid          primary key default gen_random_uuid(),
  wallet_id         uuid          not null references wallets(id) on delete cascade,
  recurring_rule_id uuid          references recurring_rules(id),
  amount            numeric(10,2) not null,
  type              text          not null,
  date              date          not null,
  name              text,
  note              text,
  remark            text,
  is_confirmed      boolean       not null default false,
  completed_at      timestamptz,
  created_at        timestamptz   not null default now()
);
```

3.4.3 Columns

wallet_id references the wallet this transaction belongs to. The `on delete cascade` means: if a wallet is deleted, all its transactions are deleted with it. A wallet and its entire financial history live and die together.

recurring_rule_id links a fixed-wallet debit to the recurring rule that generated it. It is nullable: variable wallet debits (manual cost entries) have no rule, and all credit transactions (income distributions) have no rule either. For a fixed-wallet payment, this column is how the application knows which rule was fulfilled when filtering the payment history or determining which upcoming payments remain pending.

amount is always positive, regardless of whether the transaction is a credit or debit. The direction is stored separately in `type`.

type is either `debit` (money leaving the wallet) or `credit` (money entering it). Credits arise from income distribution. Debits arise from fixed-wallet payment confirmations and variable-wallet manual cost entries.

date is the calendar date the transaction is attributed to, as a `date` type (no time component). For

income distribution transactions it is the date the income was entered. For fixed-wallet confirmations it is the payment date. For variable-wallet entries it is the date the user assigns to the transaction.

name is a free text label for the transaction, used mainly by variable-wallet entries to describe what was purchased. Fixed-wallet transactions derive their label from the recurring rule name rather than storing it redundantly here.

note and **remark** are both optional text fields that serve different purposes. `note` is set programmatically: income distribution transactions receive a standardised note such as `Income distribution - Salary`. `remark` is set by the user at the moment of confirming a fixed-wallet payment and represents a personal annotation about that specific instance.

is_confirmed distinguishes between a transaction that has been acknowledged by the user and one that is pending. For fixed wallets, the confirm handler sets this to true and calls `decrement_wallet_balance` at the same moment. Variable-wallet entries are inserted as confirmed immediately, since the user's act of submitting the form is itself the confirmation.

completed_at records the exact timestamp when confirmation happened. It differs from `date` (the attributed date, which the user chooses) and from `created_at` (when the row was inserted): `completed_at` is specifically when the user performed the act of confirming.

3.4.4 Foreign key relationships

`wallet_id` references `wallets(id)` with cascade on delete. `recurring_rule_id` references `recurring_rules(id)` without cascade: if a recurring rule is retired (its `end_date` is set), the past transactions that referenced it remain intact and the foreign key still points to the archived rule row. A transaction never references `income_entries` directly; the connection is implicit through the note text.

3.4.5 Design notes

Storing `amount` as always positive and recording direction in a separate `type` column rather than using signed amounts (positive for credit, negative for debit) makes aggregations more readable. Summing all credits for a period and summing all debits are two separate, self-documenting queries. A signed-amount design would require careful sign-handling to avoid conflating the two directions.

3.5 Table: recurring_rules

3.5.1 What the table represents

Each row of the `recurring_rules` table defines a recurring payment schedule for a fixed wallet: its name, amount, and when it falls due. The `TransactionChecklist` and `UpcomingPayments` components generate their content entirely from this table, using the date-calculation logic in `recurringUtils.js` to project future payment dates from the stored schedule. Rows in this table are never deleted; retiring a rule means setting its `end_date`.

3.5.2 Schema

```
create table recurring_rules (
  id                  uuid          primary key default gen_random_uuid(),
  wallet_id           uuid          not null references wallets(id),
  name                text          not null,
  description         text,
  amount              numeric(10,2) not null,
  frequency           text          not null,
  day_of_month        integer,
  quarter_month       integer,
  yearly_month        integer,
  custom_dates        jsonb,
  custom_cycle_years  integer,
  start_date          date          not null,
  end_date            date,
  parent_rule_id      uuid          references recurring_rules(id),
  created_at          timestamptz   not null default now()
);
```

3.5.3 Columns

wallet_id references the fixed wallet this rule belongs to. A wallet may have many rules: a household-costs wallet might hold rent, utilities, and insurance as three separate recurring rules.

frequency takes one of six values: `daily`, `weekly`, `monthly`, `quarterly`, `yearly`, or `custom`. Each value triggers a different calculation path in `recurringUtils.js`.

day_of_month carries different meanings depending on frequency. For monthly rules it is the day number within the month (1 through 31). For weekly rules it encodes the day of the week as an integer where 1 is Monday and 7 is Sunday. The overloading of one column for two semantics is deliberate: separate `day_of_month` and `day_of_week` columns would each be null on all rows that did not need them, and a single nullable column reduces width without ambiguity, because the `frequency` column makes the interpretation unambiguous.

quarter_month is used when frequency is `quarterly`. It holds 1, 2, or 3, identifying which month within the quarter the payment falls. Combined with `day_of_month`, it pinpoints the exact day: `quarter_month = 1` and `day_of_month = 15` means the 15th of January, April, July, and October.

yearly_month is used when frequency is `yearly` and holds the zero-indexed month number (0 for January, 11 for December), following JavaScript's `Date` convention. `recurringUtils.js` projects forward from `start_date` using this value.

custom_dates is a JSONB column holding an array of date strings in `MM-DD` format (for example, `["03-01", "09-01"]`) for payments with an irregular schedule that repeats on the same calendar

dates each year. JSONB is PostgreSQL's binary JSON type: it is stored in a parsed binary form rather than raw text, which allows the database to validate the JSON structure and permits queries into the array contents. For the application, `custom_dates` is read as a JavaScript array.

custom_cycle_years accompanies `custom_dates` and allows the cycle to span multiple years. A value of 2 means the dates fire every two years rather than annually.

start_date is the date the rule began. The application uses it as the lower bound when generating the payment schedule: no dates before `start_date` are projected. For a rent contract that began on 1 March 2024, `start_date` is `2024-03-01`.

end_date is null for active rules and set to the final date of applicability for retired ones. The application queries `end_date is null` to find currently active rules.

parent_rule_id is a self-referencing foreign key. When a rule is edited (for example, the rent amount increases), the application does not update the existing row. Instead, it sets `end_date` on the old row and inserts a new row with `parent_rule_id` pointing at the old one. This preserves the complete history of a rule across amount changes. The chain of `parent_rule_id` links forms a linked list from the most recent version back to the original. An equivalent chain exists on `income_recurring` (section 3.7) for the salary growth chart.

3.5.4 Foreign key relationships

`wallet_id` references `wallets(id)` without cascade on delete: the application guards at the application layer against deleting a wallet that still has active rules. `parent_rule_id` references `recurring_rules(id)` on the same table (a self-join). `transactions.recurring_rule_id` references this table from the other side.

3.5.5 Design notes

The variety of scheduling columns might seem verbose, but the payment world genuinely demands all of them. Rent is monthly on a fixed day. A quarterly insurance premium needs `quarter_month`. A yearly car tax needs `yearly_month`. A union membership with specific annual dates needs `custom_dates`. Rather than store a single opaque expression string that `recurringUtils.js` would have to parse, the schema stores each parameter of the schedule in a typed column the application can query and manipulate directly.

3.6 Table: income_entries

3.6.1 What the table represents

Each row of the `income_entries` table records one income event: an amount of money that entered the system. A row is inserted whenever income is logged, regardless of whether it came via a quick manual entry, a recurring income schedule, or a template. The income history page on the Income tab reads from this table.

3.6.2 Schema

```
create table income_entries (
  id                  uuid          primary key default gen_random_uuid(),
  amount              numeric(10,2) not null,
  source              text          not null,
  source_type         text          not null,
  date                date          not null,
  note                text,
  completed_at        timestamptz,
  income_recurring_id uuid          references income_recurring(id),
  income_template_id  uuid          references income_templates(id),
  created_at          timestamptz   not null default now()
);
```

3.6.3 Columns

source is the display name shown in the income history: the name of the recurring income (for example, "Salary"), the template name (for example, "Freelance project"), or a user-supplied description for quick entries.

source_type records how the entry was created: `manual` for a quick one-off entry, `recurring` for an instance of a recurring income schedule, or `template` for an instance of a named template. This column is used both for display (the income history shows a type badge per row) and for understanding the provenance of the entry.

date is the income date assigned by the user, as a `date` type. For recurring income it is usually today but may be adjusted. For manual entries it is whatever date the user selects.

note is an optional free-text field the user may fill in at entry time.

completed_at records the timestamp of entry creation, parallel in purpose to the same column on transactions.

income_recurring_id links this entry to its recurring income definition when `source_type` is `recurring`. It is null otherwise. The link allows the application to group entries by their source when displaying the history of a particular recurring income.

income_template_id links this entry to its template definition when `source_type` is `template`. It is null otherwise. Quick entries leave both foreign keys null; they are self-contained.

3.6.4 Foreign key relationships

Both `income_recurring_id` and `income_template_id` are nullable foreign keys without cascade. Deleting a recurring income definition or a template does not remove past income entries that used

them; the entries are permanent history and retain their `source` text even if the originating record is gone.

3.6.5 Design notes

`income_entries` does not store how the income was distributed across wallets. Distribution produces credit transactions in the `transactions` table; the income entry is the cause and those transactions are the effects. Tracing the full story of one income entry means reading the entry row and then finding all credit transactions inserted on the same date with a matching source name in their `note` field. A direct foreign key from `transactions` back to `income_entries` would make this join precise; the current design traces the relationship through the note text, which is a pragmatic choice given that the distribution was originally built without that link.

3.7 Table: income_recurring

3.7.1 What the table represents

Each row of the `income_recurring` table defines a recurring income source: a salary, a pension, a regular freelance retainer. Like `recurring_rules`, records here are never hard-deleted. Retiring an income source means setting its `end_date`. Changing its amount means archiving the current row and creating a new one with `parent_rule_id` pointing backward, so the full history of salary changes is preserved.

3.7.2 Schema

```
create table income_recurring (
  id             uuid          primary key default gen_random_uuid(),
  name           text          not null,
  amount         numeric(10,2) not null,
  frequency      text          not null,
  day_of_month   integer,
  start_date     date          not null,
  end_date       date,
  parent_rule_id uuid          references income_recurring(id),
  created_at     timestamptz   not null default now()
);
```

3.7.3 Columns

name is what the user sees: “Salary”, “Pension”, “Freelance retainer”. It is stored in any income entry created from this record as the `source` field.

amount is the expected income amount for each occurrence of this recurring source.

frequency is one of: `daily`, `weekly`, `monthly`, `quarterly`, or `yearly`. The `custom` frequency supported by `recurring_rules` is absent here, because income schedules are generally regular; the application does not yet need to project future income for custom-date patterns.

day_of_month carries the same semantics as in `recurring_rules`: the day within the month for monthly frequency, or the weekday number for weekly frequency.

start_date is when the income source began, used as the lower bound for projecting past and future occurrences.

end_date is null for an active income source and set when it is retired.

parent_rule_id is a self-reference, exactly as in `recurring_rules`. The salary growth chart in `IncomeRecurringDetail.jsx` follows this chain to plot amount changes over time as a step chart: each version of the record contributes one step on the chart, from its `start_date` to its `end_date`.

3.7.4 Foreign key relationships

`parent_rule_id` references `income_recurring(id)` on the same table. `income_entries.income_recurring_id` references this table, linking each instance of income logging back to its source definition. `income_distribution_rules.income_recurring_id` (section 3.9) also references this table, linking the saved wallet distribution plan to its income source.

3.8 Table: income_templates

3.8.1 What the table represents

Each row of the `income_templates` table represents a named, reusable income preset: a saved name and default amount for income sources that are not on a fixed schedule but recur occasionally under the same label. A freelance project, a tax refund, a gift: the user creates a template once and reuses it so they do not have to retype the same label each time.

3.8.2 Schema

```
create table income_templates (
  id         uuid          primary key default gen_random_uuid(),
  name       text          not null,
  amount     numeric(10,2) not null,
  note       text,
  created_at timestamptz   not null default now()
);
```

3.8.3 Columns

name is the label shown on the template card on the income page and used as the `source` field in any income entry created from the template.

amount is the default amount, which the user may override at entry time. A template for "Freelance project" might carry a default of €500 but be overridden to €750 for a larger engagement.

note is an optional stored annotation visible when using the template, providing context or a reminder.

3.8.4 Foreign key relationships

`income_entries.income_template_id` references this table. No other table references `income_templates`.

3.8.5 Design notes

Templates are the simplest table in the schema: four columns of meaningful data, no self-references, and no soft-deletion. They can be deleted outright because deleting a template does not remove the income entries that were created from it; those entries retain their `source` text and `amount` and remain in the history regardless of whether the template still exists.

3.9 Table: income_distribution_rules

3.9.1 What the table represents

Each row of the `income_distribution_rules` table encodes one line of a recurring income's wallet distribution: how much of that income should flow to a specific wallet each time it is logged. If a salary of €2,500 is to be split into €900 for Rent, €400 for Variable Expenses, €600 for Savings, and €600 for Unallocated, four rows exist for that recurring income, one per target wallet. Together those four rows constitute the complete, saved distribution plan for that income source.

3.9.2 Schema

```
create table income_distribution_rules (
  id                  uuid          primary key default gen_random_uuid(),
  income_recurring_id uuid          not null references income_recurring(id) on delete cascade,
  wallet_id           uuid          not null references wallets(id) on delete cascade,
  amount              numeric(10,2) not null,
  priority            integer       not null default 0,
  created_at          timestamptz   not null default now()
);
```

3.9.3 Columns

income_recurring_id links the rule set to a specific recurring income. All rows sharing the same `income_recurring_id` together represent the complete distribution for that income source.

When the application processes a recurring income entry, it reads all rows with the matching `income_recurring_id` and passes them to `distributeIncome.js`.

wallet_id identifies which wallet receives the money. The distribution popup shows all active non-system wallets; each one the user assigns an amount to becomes a row here.

amount is a fixed euro amount, not a percentage. The constraint that the amounts across all rows for one recurring income must sum exactly to the income amount is enforced by the application at the moment of saving the distribution setup, not by a database constraint.

priority controls the order in which distributions are applied when income is received. Lower numbers are applied first. This matters for capped wallets: if a capped wallet receives its allocation before others and its cap is already reached, the overflow goes to Unallocated. Priority ordering ensures predictable behaviour when multiple wallets with caps interact with the same pool of money.

3.9.4 Foreign key relationships

Both foreign keys carry `on delete cascade`. If a recurring income is deleted, all its distribution rules are deleted with it, because the rule set has no meaning without the income it belongs to. If a wallet is deleted, all distribution rules that assigned money to it are deleted too, because the rule set would be incomplete and unexecutable.

3.9.5 Design notes

Distribution rules for manual entries and templates are not stored in this table. When a user performs a manual or template income entry, the distribution is decided interactively through the `DistributionPopup` component. The chosen amounts are applied immediately through `distributeIncome.js`, producing credit transactions, but no row is written to `income_distribution_rules`. Only recurring income has a saved, repeatable distribution plan. For all other income types, the distribution decision is made fresh each time.

3.10 Table: budget_allocations

3.10.1 What the table represents

Each row of the `budget_allocations` table records one historical budget assignment for a wallet: how much was allocated to it during a specific period. The table forms a timeline of budget changes, where each row is one version of the budget, valid from `valid_from` until `valid_until`. The current allocation is the row whose `valid_until` is null.

3.10.2 Schema

```
create table budget_allocations (
  id          uuid          primary key default gen_random_uuid(),
  wallet_id   uuid          not null references wallets(id),
```

```
  amount      numeric(10,2) not null,
  valid_from  date          not null,
  valid_until date,
  created_at  timestamptz   not null default now()
);
```

3.10.3 Columns

wallet_id links the allocation to a wallet.

amount is the budget figure for this period, using the same two-decimal precision as all other monetary columns.

valid_from is the first date this budget applies. When a user raises the budget of a variable wallet from €300 to €400 starting next month, a new row is inserted with `valid_from` set to the first of next month.

valid_until is the last date this budget applied, or null if the allocation is still current. When a new allocation is created for a wallet, `valid_until` is set on the outgoing row to the day before the new one begins, so the timeline has no gaps or overlaps.

3.10.4 Foreign key relationships

`wallet_id` references `wallets(id)` without cascade. No other table references `budget_allocations`.

3.10.5 Design notes

The current application reads the wallet's budget from the `wallets.budget` column directly for display and calculations, rather than querying `budget_allocations` on every page load. The `budget_allocations` table exists to support historical analysis: knowing what the budget was during a past period, so that overspend can be evaluated correctly in retrospect. The dashboard's performance calculations may draw on it over time. Because of this dual storage, a write to `wallets.budget` should always be accompanied by a corresponding write to `budget_allocations`.

One detail about lifecycle: the Settings page's "Delete all data" action removes rows from `budget_allocations` along with `transactions` and `income_entries`, treating historical budget records as part of the data that gets cleared rather than part of the structural configuration that persists (see ARCHITECTURE.md, Settings Page section).

3.11 Table: settings

3.11.1 What the table represents

The `settings` table holds exactly one row: the application-wide configuration for the current user. Unlike every other table, it is not a collection of entities but a single record. It is read once on startup and written only when the user changes a preference on the Settings page.

3.11.2 Schema

```
create table settings (
  id                  uuid        primary key default gen_random_uuid(),
  currency            text        not null default 'EUR',
  currency_symbol     text        not null default '€',
  month_start_day     integer     not null default 1,
  theme               text        not null default 'light',
  strict_distribution boolean     not null default true,
  created_at          timestamptz not null default now()
);
```

3.11.3 Columns

currency and **currency_symbol** store the user's currency preference. In the current single-user implementation these default to EUR and the euro sign and are read-only in the interface. They exist as columns rather than hardcoded constants so that multi-currency support would require only a settings update rather than a code change.

month_start_day is the day of the month considered the start of a financial month. For users paid on the 25th, setting this to 25 means that "this month's" income and spending are measured from the 25th to the 24th, aligning the reporting window with the natural rhythm of their cash flow. The dashboard calculations in `dashboardCalcs.js` read this value when computing monthly summaries and projected positions.

theme is either `light` or `dark` and controls the CSS class applied to the root element, which selects the active Tailwind colour scheme.

strict_distribution governs the distribution popup for manual and template income entries. When true, the user must assign the full income amount before confirming; leaving any amount unallocated is prevented. When false, any unallocated remainder is automatically routed to the Unallocated wallet on confirmation. Recurring income always uses strict distribution regardless of this setting, because its rules are pre-configured to sum exactly to the income amount.

3.11.4 Foreign key relationships

The `settings` table references no other table and is referenced by no other table.

3.11.5 Design notes

The single-row design is a valid and common pattern for configuration tables in small applications. Because there is only one row, the query that reads it omits any filter: `select * from settings limit 1`. When multi-user support is added, a `user_id` column will be introduced and the query will filter by `auth.uid()`, at which point each user will have their own settings row and the table will become a proper collection like the others.

3.12 The two balance functions

3.12.1 Why they exist

Section 2.7.6 introduced database functions and the reason for using them: atomicity. A wallet balance change involves reading the current balance and writing a new value. If two requests arrive at nearly the same moment, a naive approach would be:

1. Request A reads balance: €450.
2. Request B reads balance: €450.
3. Request A writes €450 + €200 = €650.
4. Request B writes €450 + €300 = €750.

The final balance is €750, but it should be €950. Both increments happened, but B's write overwrote A's, and €200 is permanently lost. This class of problem is called a race condition and it occurs whenever a read and a write are not performed as a single indivisible step.

The solution is to push the update inside the database as an `update ... set balance = balance + amount` statement. The database engine performs the read and write in one atomic operation; no other update can slip between them. That is the entire content of the two functions.

3.12.2 Function definitions

```
create or replace function increment_wallet_balance(
  p_wallet_id uuid,
  p_amount    numeric
)
returns void
language sql
as $$
  update wallets
  set    balance = balance + p_amount
  where  id = p_wallet_id;
$$;

create or replace function decrement_wallet_balance(
  p_wallet_id uuid,
  p_amount    numeric
)
returns void
language sql
as $$
  update wallets
  set    balance = balance - p_amount
```

```
  where  id = p_wallet_id;
$$;
```

Both functions take the same two parameters: `p_wallet_id` identifies the row to update, and `p_amount` is always a positive number representing the magnitude of the change. The `p_` prefix is a naming convention signalling that these are procedure parameters, not column names, to avoid confusion when they appear alongside column names in the SQL body. The functions return `void`, meaning they perform a side effect and return no data. They are written in the `sql` language variant, meaning the body is pure SQL with no procedural logic.

3.12.3 How they are called from JavaScript

The frontend never updates the `wallets.balance` column directly. Every balance change anywhere in the application goes through one of these two functions via `supabase.rpc`:

```
// crediting a wallet during income distribution (from distributeIncome.js)
await supabase.rpc('increment_wallet_balance', {
  p_wallet_id: dist.wallet_id,
  p_amount: amount,
})

// debiting a wallet when a fixed payment is confirmed (from WalletDetail.jsx)
await supabase.rpc('decrement_wallet_balance', {
  p_wallet_id: walletId,
  p_amount: item.rule.amount,
})
```

`supabase.rpc` sends the function name and its parameters as an HTTP request to the Supabase API (see section 2.7.6). Supabase executes the function on the database server. Because the update happens inside the database engine, RLS policies apply (section 3.13) and the operation is atomic.

3.13 Row Level Security

3.13.1 What RLS does

Section 2.9 described the four security layers of the application. The fourth and decisive layer is Row Level Security: policies stored inside the database that are evaluated on every query, regardless of how the request arrived. Even a user who takes the anon key (the key visible in any browser's network traffic, as section 2.9 explains as intentional and safe) and constructs raw HTTP requests cannot read or write data if the RLS policies reject the request.

3.13.2 Current policies

RLS is enabled on all nine tables. The current policy on every table follows this pattern:

```
alter table wallets enable row level security;

create policy "authenticated users only"
  on wallets
  for all
  using (auth.role() = 'authenticated');
```

The same two statements apply to each of the remaining eight tables, with the table name substituted. `for all` means the policy governs every operation: select, insert, update, and delete. `auth.role()` is a Supabase function that returns the role of the caller. It returns `authenticated` when the request carries a valid session token and `anon` otherwise. The `using` clause must evaluate to true for a row to be accessible; any row where it evaluates to false is invisible and unwriteable, as though it did not exist.

3.13.3 What the current policy enforces

The current policy enforces that only a logged-in user may access any data at all. An unauthenticated request (no session token, or an expired one) receives an empty result set or an error on every query, even with a valid anon key. This closes the gap between the frontend auth guard (the application redirects unauthenticated visitors to the login page) and the data layer: the data cannot be extracted by bypassing the frontend code.

3.13.4 The current limitation and the planned tightening

The current policy checks whether someone is logged in, not who is logged in. With a single user account, these two checks are equivalent: the only person who can log in is the owner. The moment a second account exists, any authenticated user would see all rows belonging to any other user.

Multi-user support, planned after Phase 7, will add a `user_id uuid references auth.users` column to every table and replace the current policy with:

```
create policy "users see own rows only"
  on wallets
  for all
  using (auth.uid() = user_id);
```

`auth.uid()` returns the UUID of the currently authenticated user. This policy reduces the visible rows on every query to only those owned by the requester. The anon key exposure that section 2.9 describes as safe remains safe at that stage too, because the policy runs in the database after the request arrives, not in the frontend before it leaves.

3.14 How the tables connect

The nine tables form a clear dependency graph. The central node is `wallets`: seven of the other eight tables reference it directly or indirectly. The diagram below shows which table references which (an arrow means "has a foreign key pointing to"):

```
income_recurring  (self-ref: parent_rule_id)

       income_entries

       income_distribution_rules              wallets

income_templates

       income_entries

recurring_rules  (self-ref: parent_rule_id)

              transactions

                                    wallets

budget_allocations                          wallets

settings   (no foreign keys in either direction)
```

Reading the graph: `transactions` references both `wallets` (via `wallet_id`) and `recurring_rules` (via `recurring_rule_id`). `recurring_rules` references `wallets` and itself via `parent_rule_id`. `income_distribution_rules` references both `income_recurring` and `wallets`. `income_entries` references both `income_recurring` and `income_templates`. `budget_allocations` references `wallets`. `settings` stands alone.

The practical consequence for the frontend is that queries frequently span tables. Fetching everything needed to display a wallet detail page requires `wallets`, `transactions`, and `recurring_rules`, typically in parallel (section 2.7.4). The income distribution operation touches `income_recurring`, `income_distribution_rules`, `wallets`, and `transactions` in sequence. The dependency graph makes the read and write patterns of the application predictable: the wallets table is at the centre of nearly every significant operation, which is why its balance must be protected by atomic functions (section 3.12) and why every table that references it defines clearly whether wallet deletion should cascade.

4 The codebase, script by script

This chapter walks through every file in `src/`, then the project-root configuration files, in dependency order: the foundational utilities first, then the application frame, then pages and components grouped by the feature they belong to.

4.1 The src folder

All application code lives in `src/`, divided between the four files at its root (`main.jsx`, `App.jsx`, `index.css`, and the Vite entry point), a `lib/` folder of pure utilities, a `pages/` folder of full-screen views, and a `components/` folder of reusable interface pieces.

4.1.1 The lib folder

The `lib` folder contains four pure JavaScript modules: `supabase.js`, `recurringUtils.js`, `distributeIncome.js`, and `dashboardCalcs.js`. None of them contain any JSX or render anything on screen; they provide logic and utilities that the pages and components call. Because they have no dependencies on UI components, they can be read and understood independently of the interface.

4.1.1.1 supabase.js

`supabase.js` creates the single Supabase client instance used throughout the entire application. It is three lines of meaningful code: read two environment variables, call `createClient`, and export the result. Every file that talks to the database or to Supabase's authentication system imports this one object. Creating it centrally means the connection is configured in exactly one place; nothing else in the project needs to know the project URL or the key.

Section 2.7.3 explains the role of the client object: it wraps the HTTP requests that travel from the browser to Supabase, so that queries can be written as readable JavaScript method chains rather than raw network calls. The two environment variables come from `.env.local` locally and from the Vercel dashboard in production; section 2.6.3 explains how Vite exposes them and why they never enter the repository.

Reading the environment variables. Vite makes any variable whose name begins with `VITE_` available at `import.meta.env`. The two variables needed here are the Supabase project URL and the anon key. Section 2.9 explains that the anon key is safe to expose in browser traffic: it identifies the project but grants no data access on its own.

```
const supabaseUrl     = import.meta.env.VITE_SUPABASE_URL
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY
```

Creating and exporting the client. `createClient` from the `@supabase/supabase-js` package takes the URL and the anon key and returns a configured client object. The result is exported as a named export. Any other file imports it with `import { supabase } from './lib/supabase'` (or the equivalent relative path from deeper in the folder tree). The instance is created once at module load time and shared everywhere; the application never calls `createClient` a second time.

```
export const supabase = createClient(supabaseUrl, supabaseAnonKey)
```

Imports: none (project files)

Exports: `supabase`

Used by: App.jsx, Login.jsx, Layout.jsx, Wallets.jsx, WalletDetail.jsx, Dashboard.jsx, Income.jsx, IncomeRecurringDetail.jsx, Settings.jsx, DistributionPopup.jsx, RecurringRules.jsx, TransactionChecklist.jsx, PaymentHistory.jsx, UpcomingPayments.jsx, VariableHistory.jsx, VariableOverview.jsx, VariableTransactionForm.jsx, VariableTransactionList.jsx, WalletTrendsChart.jsx, distributeIncome.js

---

4.1.1.2 recurringUtils.js

`recurringUtils.js` contains the date-generation logic for recurring schedules. Given a rule from the `recurring_rules` or `income_recurring` table (see sections 3.5 and 3.7) and a reference date or count, the functions here produce the actual calendar dates on which that rule fires. The rest of the codebase treats dates as data it receives; this file is the place where schedule definitions stored in the database become concrete lists of dates.

The file exports two main functions and one utility. `generatePaymentDates` returns every date from a rule's start up to a given cutoff boundary. `generateUpcomingDates` returns the next N dates from a given starting point. `formatFrequency` maps a frequency code to a display string. The file imports several helpers from the `date-fns` library for calendar arithmetic but no project files, and it has no side effects: it touches neither the database nor any component state.

The two public functions. `generatePaymentDates` is used by `TransactionChecklist.jsx`, `UpcomingPayments.jsx`, and `dashboardCalcs.js` to find all payment dates in a window of time. It first hands off to a dedicated path for custom schedules, then finds the first valid payment date

for the given frequency, and steps forward one period at a time until the cutoff is reached. A safety limit of 1,000 iterations prevents runaway loops on misconfigured rules.

```
export function generatePaymentDates(rule, upToDate) {
  if (rule.frequency === 'custom') return generateCustomDates(rule, upToDate)
  const start  = startOfDay(new Date(rule.start_date))
  const cutoff = startOfDay(upToDate)
  let current  = getFirstPaymentDate(rule, start)
  ...
  while (!isAfter(current, cutoff) && dates.length < limit) {
    if (!end || !isAfter(current, end)) dates.push(new Date(current))
    current = advanceDate(current, rule)
  }
  return dates
}
```

Aligning to the first payment day. Not every rule starts on its payment day. A monthly payment that falls on the 28th but whose rule was created on the 1st of March must skip ahead to the 28th before collecting dates. The private `getFirstPaymentDate` function handles this alignment through a `switch` on `rule.frequency`. The monthly case, shown below, constructs the target day in the start month and, if that date has already passed, advances by one month to find the first future occurrence.

```
case 'monthly': {
  const day = rule.day_of_month ?? 1
  let d = new Date(start.getFullYear(), start.getMonth(), day)
  if (isBefore(d, start)) d = addMonths(d, 1)
  return d
}
```

The weekly case has an extra step: JavaScript's `getDay()` uses 0 for Sunday and numbers days from there, while the database stores weekdays as 1 for Monday through 7 for Sunday (see section 3.5.3). A one-line conversion function `toMon` remaps the JavaScript value before the comparison.

Stepping forward by one period. After the first date is found, `advanceDate` moves the cursor forward by exactly one period on each iteration of the main loop. The `date-fns` functions it delegates to handle all calendar edge cases: `addMonths` from 31 January correctly returns 28 February rather than a nonexistent 31 February.

```
function advanceDate(date, rule) {
  switch (rule.frequency) {
    case 'daily':     return addDays(date, 1)
    case 'weekly':    return addWeeks(date, 1)
    case 'monthly':   return addMonths(date, 1)
    case 'quarterly': return addQuarters(date, 1)
```

```
    case 'yearly':    return addYears(date, 1)
    default:          return addMonths(date, 1)
  }
}
```

Collecting upcoming dates. `generateUpcomingDates` works in two phases. First it fast-forwards the cursor past any dates before `fromDate`, advancing one period at a time. Then it collects dates until `count` is reached or the rule's `end_date` is passed. This variant is used by `UpcomingPayments.jsx` to list what is coming next and by `dashboardCalcs.js` to project income arrivals within a 30-day window.

```
while (isBefore(current, from) && steps < limit) {
  current = advanceDate(current, rule)
  steps++
}
while (dates.length < count) {
  if (end && isAfter(current, end)) break
  dates.push(new Date(current))
  current = advanceDate(current, rule)
}
```

Custom schedules. The `custom` frequency does not follow a computed pattern. Instead, the rule's `custom_dates` column holds an explicit array of `MM-DD` strings (see section 3.5.3). The private `generateCustomDates` and `generateCustomUpcoming` functions iterate over calendar years, expanding each `MM-DD` into a concrete date, and collect those that fall within the relevant range. The `custom_cycle_years` field on the rule controls how many years elapse between repetitions of the same date list.

```
for (const mmdd of rule.custom_dates) {
  const [mm, dd] = mmdd.split('-').map(Number)
  const d = startOfDay(new Date(cycleStart, mm - 1, dd))
  if (isAfter(d, cutoff)) return dates
  if (!isBefore(d, startOfDay(new Date(rule.start_date))))
    dates.push(d)
}
cycleStart += cycleYears
```

Formatting a frequency for display. `formatFrequency` is a small lookup utility: it maps a stored frequency code such as `'monthly'` to the display string `'Monthly'`. The `??` operator (section 2.3.6) returns the original code unchanged if it is not in the map, so an unknown frequency degrades gracefully rather than showing `undefined`.

```
export function formatFrequency(frequency) {
  const map = {
```

```
    daily: 'Daily', weekly: 'Weekly', monthly: 'Monthly',
    quarterly: 'Quarterly', yearly: 'Yearly', custom: 'Custom'
  }
  return map[frequency] ?? frequency
}
```

Imports: none (project files)

Exports: `generatePaymentDates`, `generateUpcomingDates`, `formatFrequency`

Used by: TransactionChecklist.jsx, UpcomingPayments.jsx, Income.jsx, RecurringRules.jsx, dashboardCalcs.js

---

4.1.1.3 distributeIncome.js

`distributeIncome.js` exports one async function that executes the complete income distribution operation. Given a list of intended wallet allocations and some context about the income event, it credits each wallet, applies cap and reduction logic for automated distributions, routes any overflow to the Unallocated wallet, and records every credit as a transaction row. It is the most side-effect-heavy file in the lib folder: unlike `recurringUtils.js` and `dashboardCalcs.js`, it writes to the database rather than only computing values.

The function is called from `Income.jsx` and `IncomeRecurringDetail.jsx` whenever any form of income is confirmed, regardless of whether the source was a quick manual entry, a recurring schedule, or a template. It imports the `supabase` client from `supabase.js` and calls the `increment_wallet_balance` database function (see section 3.12) for every credit. Section 1.2 introduced the stored-balance model that makes these increment calls necessary; section 3.4 describes the `transactions` table that receives the resulting rows.

The function signature and the isAutomated flag. `distributeIncome` receives its arguments as a single destructured object: `distributions` (an array of `{wallet_id, amount}` pairs representing the intended split), `wallets` (the full list of active wallets, needed to read each wallet's type, budget, and balance), `unallocatedWalletId`, `sourceName`, `date`, and `isAutomated`. The last parameter is the central branch point of the entire function. When false, no cap logic applies; when true, the cap and reduction rules for automated recurring income are enforced.

```
export async function distributeIncome({
  distributions, wallets, unallocatedWalletId,
  sourceName, date, isAutomated = false
}) {
  const transactionRows = []
  for (const dist of distributions) {
```

```
    const amount = Number(dist.amount)
    if (!amount || amount <= 0) continue
    const wallet = wallets.find(w => w.id === dist.wallet_id)
```

Manual and template income: full amount, no cap check. When `isAutomated` is false, every allocation is credited in full regardless of the wallet's balance or cap state. The function calls `increment_wallet_balance` via `supabase.rpc` (section 2.7.6) and adds a transaction row to the `transactionRows` accumulator. A `continue` statement skips the rest of the loop body so the automated cap logic is never reached. This reflects the design principle that a user manually choosing where to direct money is making an intentional, overriding decision.

```
if (!isAutomated) {
  await supabase.rpc('increment_wallet_balance',
    { p_wallet_id: dist.wallet_id, p_amount: amount })
  transactionRows.push({
    wallet_id: dist.wallet_id, type: 'credit', amount,
    date, note: `Income distribution - ${sourceName}`, is_confirmed: true,
  })
  continue
}
```

Automated income into a capped wallet. When `isAutomated` is true and the wallet's `budget_type` is `'capped'`, the function enters a three-way branch based on the wallet's current state. The first case is the wallet below its cap: the credit is limited to `cap - balance` so the balance is never pushed above the ceiling, and any surplus becomes `excess` that is credited to the Unallocated wallet instead. The note on the overflow transaction identifies which wallet caused it.

```
if (balance < cap) {
  const creditToWallet = Math.min(amount, cap - balance)
  const excess         = Number((amount - creditToWallet).toFixed(2))
  await supabase.rpc('increment_wallet_balance',
    { p_wallet_id: wallet.id, p_amount: creditToWallet })
  if (excess > 0 && unallocatedWalletId) {
    await supabase.rpc('increment_wallet_balance',
      { p_wallet_id: unallocatedWalletId, p_amount: excess })
  }
}
```

The second case is the wallet already at or above its cap with `cap_reduction_enabled` set to true. Rather than redirecting the entire allocation, the wallet continues receiving a reduced fraction of it. `cap_reduction_rate` is a decimal between 0 and 1 (section 3.3.3); multiplying the allocation by this rate gives `reducedAmount`, while the remainder becomes `excess` for Unallocated. The third case is the wallet at cap with reduction disabled: the full allocation routes directly to Unallocated and the wallet receives nothing.

```
} else if (wallet.cap_reduction_enabled) {
  const rate          = Number(wallet.cap_reduction_rate)
  const reducedAmount = Number((amount * rate).toFixed(2))
  const excess        = Number((amount - reducedAmount).toFixed(2))
  if (reducedAmount > 0) {
    await supabase.rpc('increment_wallet_balance',
      { p_wallet_id: wallet.id, p_amount: reducedAmount })
  }
```

Non-capped wallets and the final batch insert. Wallets that are not of `budget_type 'capped'` (fixed wallets, accumulating wallets, and the Unallocated wallet itself) receive their full automated allocation without any cap check. After all distributions have been processed and every credit call has completed, the accumulated `transactionRows` array is inserted into the `transactions` table in a single batch. One network round trip inserts all rows rather than one per distribution.

```
if (transactionRows.length > 0) {
  await supabase.from('transactions').insert(transactionRows)
}
```

Imports: `supabase` from `./supabase`

Exports: `distributeIncome`

Used by: Income.jsx, IncomeRecurringDetail.jsx

4.1.1.4 dashboardCalcs.js

`dashboardCalcs.js` is the calculation engine for the dashboard page. It contains eight exported functions that receive raw data from the database (wallets, transactions, income entries, recurring rules, distribution rules) and compute the numbers the dashboard displays: the projected 30-day cash position, the six-month outlook strip, the three alert categories, the current month's performance metrics compared against a historical baseline, and the data series consumed by the charts. None of the functions write to the database or touch any React state; they accept arrays as arguments and return plain objects or arrays of objects. This separation means the entire dashboard calculation logic can be understood and reasoned about independently of the interface.

The file imports `generatePaymentDates` and `generateUpcomingDates` from `recurringUtils.js`, and several date functions from `date-fns`. It is imported only by `Dashboard.jsx`.

Private helper functions. Two short private functions at the top of the file are called by several of the exported functions below. `isActive` checks whether a rule should be considered right now: a rule without an `end_date` is always active; one whose `end_date` has already passed is not.

`findTransaction` looks up an existing transaction for a specific rule on a specific date string, used to determine whether a scheduled payment has already been confirmed.

```
const isActive = (rule, today) =>
  !rule.end_date || !isBefore(new Date(rule.end_date), today)

function findTransaction(transactions, ruleId, dateStr) {
  return transactions.find(
    t => t.recurring_rule_id === ruleId && t.date === dateStr
  )
}
```

A third private helper, `unpaidUpcomingCosts`, iterates over all active recurring rules, generates their payment dates up to a cutoff, and sums the amounts for any payments that have not yet been confirmed. An optional `walletId` parameter scopes the sum to a single wallet, which lets the same loop serve both the total upcoming cost calculation and the per-wallet underfunding check without duplicating code.

The 30-day projected cash position. `calculateProjectedCash` is the headline number on the dashboard. The formula is: cash right now plus income expected in the next 30 days minus unpaid fixed costs due in the next 30 days. `cashNow` is the sum of all wallet balances, read directly from the stored `balance` column (section 3.3.3 explains why the balance is stored rather than computed on demand). `expectedIncome` counts upcoming occurrences of active income recurring rules within the window and multiplies by each rule's amount. `upcomingCosts` is the result of `unpaidUpcomingCosts`.

```
const cashNow = wallets.reduce((s, w) => s + Number(w.balance), 0)
let expectedIncome = 0
for (const rule of incomeRecurring) {
  if (!isActive(rule, today)) continue
  const dates = generateUpcomingDates(rule, today, 60)
    .filter(d => !isAfter(d, cutoff))
  expectedIncome += dates.length * Number(rule.amount)
}
const upcomingCosts = unpaidUpcomingCosts(recurringRules, transactions, today, cutoff)
return { cashNow, expectedIncome, upcomingCosts, projected: cashNow + expectedIncome - upcoming
```

The six-month outlook strip. `calculateMonthOutlook` takes a single month date and returns the projected net income minus costs for that month. It calls `generatePaymentDates` on both the income recurring rules and the payment rules and counts how many scheduled events fall within the month's boundaries. This function does not consult the `transactions` table; the outlook is a forward projection, not a report of what actually happened. `Dashboard.jsx` calls it six times in sequence to build the strip of upcoming months.

```
for (const rule of incomeRecurring) {
  if (!isActive(rule, monthStart)) continue
  const dates = generatePaymentDates(rule, monthEnd)
    .filter(d => !isBefore(d, monthStart))
  income += dates.length * Number(rule.amount)
}
```

The three alert functions. The "needs attention" section of the dashboard surfaces three distinct categories of problem, each produced by its own function.

`getOverduePayments` generates all payment dates up to today for every active recurring rule, then checks each date against the transactions table. Any date without a confirmed transaction is overdue. The result is sorted by date so the most overdue items appear first.

`getOverspentWallets` examines variable wallets only. For each, it sums all confirmed debit transactions within the current month (using `filter` and `reduce` as described in section 2.3.4) and compares the total to the wallet's budget. Wallets where spending exceeds the budget appear in the result with the overage amount.

`getUnderfundedWallets` examines fixed wallets. For each wallet it calculates what is owed in the next 30 days via `unpaidUpcomingCosts`, then estimates what income will arrive in that window by reading the saved distribution rules (section 3.9) and projecting occurrences of the linked recurring income sources. If the current balance plus expected incoming income falls short of the upcoming obligation, the wallet is underfunded.

```
const shortfall = upcomingNeeded - (Number(wallet.balance) + expectedIncome)
if (shortfall > 0)
  result.push({ wallet, upcomingNeeded, expectedIncome, shortfall })
```

Monthly performance metrics. `calculateMonthMetrics` takes a month and the full transactions and income-entries arrays and returns the actual income, spending, net, and savings rate (net divided by income, expressed as a percentage) for that month. A private helper `sumInRange` filters rows to a date window and reduces them to a total, keeping the arithmetic reusable for both income entries and transaction rows with a single extra filter for type and confirmation status.

`calculateMonthlyAverage` applies `calculateMonthMetrics` to each of a list of past months and averages the results. The dashboard uses a three-month average as the comparison baseline for the current month's performance display.

```
export function calculateMonthlyAverage(months, transactions, incomeEntries) {
  const metrics = months.map(m => calculateMonthMetrics(m, transactions, incomeEntries))
  const avg = key => metrics.reduce((s, m) => s + m[key], 0) / metrics.length
  const rates = metrics.map(m => m.savingsRate).filter(r => r !== null)
  return { income: avg('income'), spending: avg('spending'), net: avg('net'),
    savingsRate: rates.length > 0 ? rates.reduce((s, r) => s + r, 0) / rates.length : null }
```

```
}
```

Reconstructing historical balances. The "over time" charts need the total cash held at the end of each past month or year. That figure is not stored anywhere; only the current balances are. The private `cashAsOf` function reconstructs a past balance by starting from the current total and reversing the net effect of every confirmed transaction dated after the target point. If total cash today is €3,000 and a net of €500 in credits arrived after the end of last month, then the balance at month-end was €2,500.

```
function cashAsOf(asOf, transactions, currentTotal) {
  const changeAfter = transactions
    .filter(t => t.is_confirmed && isAfter(new Date(t.date), asOf))
    .reduce((s, t) =>
      s + (t.type === 'credit' ? Number(t.amount) : -Number(t.amount)), 0)
  return currentTotal - changeAfter
}
```

`getHistoricalSeries` and `getYearlySeries` call `calculateMonthMetrics` and `cashAsOf` for each period in a list and assemble the arrays of `{month, income, spending, totalCash}` objects that the `IncomeSpendingChart` and `CashTrendChart` components consume.

Imports: `generatePaymentDates`, `generateUpcomingDates` from `./recurringUtils`

Exports: `calculateProjectedCash`, `calculateMonthOutlook`, `getOverduePayments`, `getOverspentWallets`, `getUnderfundedWallets`, `calculateMonthMetrics`, `calculateMonthlyAverage`, `getHistoricalSeries`, `getYearlySeries`

Used by: Dashboard.jsx

4.1.2 The frame

Before any page or feature can load, five files must do their work: one mounts React onto the HTML document, one loads the stylesheet, one manages authentication state and routing, one provides the persistent sidebar shell, and one presents the login form. These files are explained together because they wire the application as a whole; every page described in the sections that follow sits inside the structure these five establish.

4.1.2.1 main.jsx

`main.jsx` is the entry point of the application. It is the first JavaScript file Vite executes, and its only job is to mount the React component tree onto the single HTML element the browser provides. Section 2.2.1 noted that `index.html` contains an almost-empty body with one element: `<div id="root"></div>`. `main.jsx` is the file that fills it.

Mounting the application. `createRoot` from the `react-dom/client` package takes a real DOM element and hands back a React root: an object that knows how to render and update a React component tree inside that element. Calling `.render(...)` on it places the entire application inside the `root` div, and from that point onward React controls that portion of the page. The import of `./index.css` on line 3 ensures the stylesheet is loaded as part of the same bundle.

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

StrictMode. The `App` component is wrapped in `StrictMode`, a built-in React wrapper that has no visible effect in production but activates extra checks during development. Its most noticeable behaviour is running every component function and every `useEffect` setup twice, to surface side effects that are not properly cleaned up. The double-run only happens in development; the production build behaves normally.

Imports: `App` from `./App.jsx`, `./index.css`

Exports: none (entry point, not imported by anything)

Used by: nothing (invoked by Vite as the bundle entry point)

4.1.2.2 index.css

`index.css` is the global stylesheet for the application. The entire file is one line:

```
@import "tailwindcss";
```

The `@import "tailwindcss"` directive, processed by the Tailwind Vite plugin, instructs Tailwind to generate its utility classes for every class name it finds in the project's source files. Section 2.5.3 explains the mechanism: at build time, Tailwind scans the JSX files and emits only the CSS for the classes actually used. Nothing needs to be written by hand. This file is the only CSS in the project.

Imports: none

Exports: none (stylesheet, not a module)

Used by: main.jsx (imported once to include the stylesheet in the bundle)

4.1.2.3 App.jsx

`App.jsx` is the root component of the application. It does two things: it tracks whether a user session exists, and it uses that knowledge to decide what the router should render. Every page and component in the project is a descendant of `App`, which means this component runs before anything else and its session state is the gate through which all content passes.

The component imports the `supabase` client from `lib/supabase.js`, the `Layout` shell, and every page component. It uses two React Router components, `BrowserRouter` and `Routes`, that it wraps around the whole tree.

The session state and its three values. Session state is initialised to `undefined`, not `null`. This distinction is intentional. During the brief moment at startup before the `getSession` call resolves, the component does not yet know whether a session exists. `undefined` represents this third state: "not yet determined". `null` would mean "definitely no session", which would cause the app to flash to the login page and then immediately redirect back to the dashboard as the real session arrived. `undefined` allows the app to show a loading screen instead, which is what section 2.7.5 describes as the correct startup pattern.

```
const [session, setSession] = useState(undefined)
```

Watching for login and logout. The `useEffect` (section 2.4.7) fires once when `App` first mounts and does two things. The first is a one-time `getSession()` call that reads any existing session from the browser's stored token and sets the state immediately. The second is a subscription to `onAuthStateChange`, which fires every time the user logs in or out, keeping the state synchronised with reality for the lifetime of the app. The cleanup function returned by the effect cancels that subscription when the component is removed, preventing stale listeners.

```
useEffect(() => {
  supabase.auth.getSession().then(({ data: { session } }) => {
    setSession(session)
  })
  const { data: { subscription } } = supabase.auth.onAuthStateChange((_event, session) => {
    setSession(session)
  })
  return () => subscription.unsubscribe()
}, [])
```

The loading guard. While `session` is `undefined`, the component renders a full-screen loading message rather than any part of the application. This prevents the router from making a routing decision before it knows whether a session exists, which would produce an incorrect flash. Once `getSession` resolves, `session` becomes either a session object or `null`, and the guard condition is no longer met.

```
if (session === undefined) {
  return (
```

```
    <div className="min-h-screen bg-gray-50 flex items-center justify-center">
      <p className="text-gray-400">Loading...</p>
    </div>
  )
}
```

The routing structure. After the guard, the component returns a `BrowserRouter` containing a `Routes`. Two top-level routes are declared. The `/login` route renders the `Login` component when no session exists, and redirects to `/` with `Navigate` when one does: a logged-in user who navigates to `/login` is sent to the dashboard automatically. The `/*` route covers everything else. If a session exists, it renders the `Layout` shell wrapping a nested `Routes` with one route per protected page. If no session exists, it redirects to `/login`. The `replace` prop on both `Navigate` calls means the redirect replaces the current entry in the browser's history rather than adding to it, so pressing the back button does not loop between the redirect and the destination.

```
<Route path="/*" element={
  session
    ? <Layout>
        <Routes>
          <Route path="/"                    element={<Dashboard />}            />
          <Route path="/wallets"             element={<Wallets />}              />
          ...
          <Route path="/income/recurring/:id" element={<IncomeRecurringDetail />} />
          <Route path="/settings"            element={<Settings />}             />
        </Routes>
      </Layout>
    : <Navigate to="/login" replace />
} />
```

The nested `Routes` inside `Layout` is a React Router v6 pattern for nested routing: the outer `Routes` matches the `/*` wildcard, and the inner `Routes` then matches the specific path among the protected pages. The `Layout` component receives the matched page as its `children` prop and renders it inside the main content area, as section 4.1.2.4 explains.

Imports: `supabase` from `./lib/supabase`, `Layout`, `Dashboard`, `Wallets`, `WalletDetail`, `Income`, `IncomeRecurringDetail`, `Settings`, `Login`

Exports: `App` (default)

Used by: main.jsx

4.1.2.4 Layout.jsx

`Layout.jsx` is the persistent shell that surrounds every protected page. It renders the sidebar, including the application title, navigation links, and sign-out button, alongside a scrollable main content area. Every page component in `pages/` is rendered inside this shell; the shell itself never changes as the user navigates between pages. Only the content area re-renders.

The component imports `NavLink` and `useNavigate` from React Router, and the `supabase` client for the sign-out operation.

The navigation items. The four navigation destinations are declared as a module-level constant, an array of objects each carrying a `path` and a `label`. Keeping this array outside the component function means it is created once, not on every render. Adding a new page to the navigation is a matter of adding one object to this array.

```
const navItems = [
  { path: '/',          label: 'Dashboard' },
  { path: '/wallets',   label: 'Wallets'   },
  { path: '/income',    label: 'Income'    },
  { path: '/settings',  label: 'Settings'  },
]
```

Sign-out. `handleSignOut` calls `supabase.auth.signOut()`, which clears the stored session token. When it resolves, `navigate('/login')` sends the browser to the login page. The `onAuthStateChange` listener registered in `App.jsx` will also fire at this moment, setting `session` to `null` and causing the router to redirect to `/login` independently. The explicit `navigate` call and the listener-driven redirect are both present for robustness; in practice the effect is the same.

```
async function handleSignOut() {
  await supabase.auth.signOut()
  navigate('/login')
}
```

The flex layout. The outer container is a full-height flex row (section 2.5.2): the sidebar is a fixed-width column on the left, and the main content area stretches to fill the remaining horizontal space. The Tailwind classes `flex h-screen` on the outer div establish the row at full viewport height; `w-56` fixes the sidebar at 224 pixels; `flex-1` on the main element absorbs all remaining width. `overflow-auto` on the main element means the page content can scroll while the sidebar stays fixed.

```
<div className="flex h-screen bg-gray-50">
  <aside className="w-56 bg-white border-r border-gray-200 flex flex-col">
    ...
  </aside>
  <main className="flex-1 overflow-auto">
    <div className="p-8">{children}</div>
```

```
  </main>
</div>
```

NavLink and active styling. React Router's `NavLink` component is like a plain `Link` but it passes an `isActive` boolean into its `className` prop, indicating whether its destination is the current page. The `Layout` component uses this to toggle between two Tailwind class strings: an indigo-tinted style for the active link and a neutral grey style for inactive ones. The callback form of `className`, `({ isActive }) => ...`, is the standard React Router v6 pattern for this.

```
<NavLink
  to={item.path}
  end={item.path === '/'}
  className={({ isActive }) =>
    `block px-3 py-2 rounded-lg text-sm font-medium transition-colors ${
      isActive ? 'bg-indigo-50 text-indigo-600' : 'text-gray-600 hover:bg-gray-100'
    }`
  }
>
```

The `end` prop on the Dashboard link deserves attention. Without it, a `NavLink` with `to="/"` would be considered active on every URL, because every path begins with `/`. The `end` prop tells React Router to match the path only when the URL is exactly `/` rather than anything that starts with it. The expression `end={item.path === '/'}` evaluates to `true` for the Dashboard entry and `false` for all others, so only the Dashboard link gets this exact-match behaviour.

The children prop. `Layout` accepts a `children` prop (section 2.4.4). In the JSX returned by `App.jsx`, the nested `Routes` tree sits between the opening and closing `<Layout>` tags, which makes it `children`. The `{children}` expression inside `Layout`'s `main` element is where the currently matched page component appears. When the user navigates from Dashboard to Wallets, only that expression's content changes; the sidebar remains.

Imports: `supabase` from `../lib/supabase`

Exports: `Layout` (default)

Used by: App.jsx

4.1.2.5 Login.jsx

`Login.jsx` renders the login form. It is the only page that is accessible without a session. On a successful login, no navigation code runs in `Login.jsx` itself: once `signInWithPassword` resolves, Supabase issues a session, the `onAuthStateChange` listener in `App.jsx` fires, the session state changes,

and the router sends the user to the dashboard automatically. The login page only has to call the API; the redirect is a consequence of the session state machinery in `App.jsx`.

Section 2.7.5 describes the authentication sequence in detail. Section 2.10 traces the full login event through every layer of the application.

State. Four pieces of state cover the form completely. `email` and `password` hold the current field values. `error` holds an error message to display or `null` when there is no error. `loading` tracks whether a request is in flight. The pattern of four `useState` declarations at the top of a component, each representing one piece of independently changing data, is the standard React form approach from section 2.4.5.

```
const [email,    setEmail]    = useState('')
const [password, setPassword] = useState('')
const [error,    setError]    = useState(null)
const [loading,  setLoading]  = useState(false)
```

The login handler. `handleLogin` is an `async` function (section 2.7.4) that runs when the user clicks the button or presses Enter. It sets `loading` to `true` and clears any previous error before calling `supabase.auth.signInWithPassword`. If the call returns an error, the message is stored in the `error` state and displayed in the form. `loading` is set back to `false` in either outcome so the button returns to its normal state.

```
async function handleLogin() {
  setLoading(true)
  setError(null)
  const { error } = await supabase.auth.signInWithPassword({ email, password })
  if (error) {
    setError(error.message)
  }
  setLoading(false)
}
```

Controlled inputs. Both form fields are controlled inputs: the React state variable drives the `value` attribute, and an `onChange` handler updates the state on every keystroke (section 2.4.5). The `onKeyDown` handler uses the `&&` shortcut from section 2.3.6 to call `handleLogin` only when the pressed key is Enter, giving the form keyboard accessibility without additional complexity.

```
<input
  type="email"
  value={email}
  onChange={e => setEmail(e.target.value)}
  onKeyDown={e => e.key === 'Enter' && handleLogin()}
  placeholder="you@example.com"
  className="w-full px-3 py-2 border border-gray-300 rounded-lg text-sm
```

```
            focus:outline-none focus:ring-2 focus:ring-indigo-500"
/>
```

Conditional error display. The error box is rendered only when `error` is not `null`, using the `&&` conditional rendering pattern from section 2.4.3. Supabase's error messages are short enough to display directly; no translation layer is needed.

```
{error && (
  <div className="bg-red-50 text-red-600 text-sm px-4 py-3 rounded-lg mb-4">
    {error}
  </div>
)}
```

The submit button. The button's `disabled` attribute is bound to the `loading` state. While a request is in flight, the button is non-interactive and rendered at half opacity via Tailwind's `disabled:opacity-50` class. The label switches between `'Sign in'` and `'Signing in...'` using a ternary expression embedded in JSX.

```
<button
  onClick={handleLogin}
  disabled={loading}
  className="w-full bg-indigo-600 text-white py-2 rounded-lg text-sm
             font-medium hover:bg-indigo-700 disabled:opacity-50"
>
  {loading ? 'Signing in...' : 'Sign in'}
</button>
```

Imports: `supabase` from `../lib/supabase`

Exports: `Login` (default)

Used by: App.jsx

4.1.3 The wallets feature

Wallets are the core organising unit of the application, as section 1.2 establishes. This group covers the pages and components that let users create, view, and manage them: the list page, the detail page, the modal for creating and editing, and every sub-component rendered inside the detail view. Because fixed and variable wallets behave entirely differently, the codebase reflects that split with dedicated sub-components for each type.

4.1.3.1 pages/Wallets.jsx

`Wallets.jsx` is the wallet list page. It loads all wallets ordered by `sort_order`, partitions them into four groups (fixed, variable, investment, and system), and renders each group under its own heading using `WalletCard`. Two overlapping modals live in the same JSX tree: a create/edit modal that opens `WalletModal` and a delete confirmation dialog assembled inline. The page follows the standard skeleton from section 2.4.8: state at the top, a `useEffect` triggering the initial load, and JSX that renders whatever state currently holds.

State and initial load. Five state variables govern the page: the wallet list, a loading flag, a boolean controlling the modal, the wallet being edited (or `null` for a fresh creation), and the wallet targeted for deletion. The `useEffect` with an empty dependency list (section 2.4.7) fires the initial `fetchWallets` call once when the page mounts.

```
const [wallets,      setWallets]      = useState([])
const [modalOpen,    setModalOpen]    = useState(false)
const [editWallet,   setEditWallet]   = useState(null)
const [deleteTarget, setDeleteTarget] = useState(null)
```

Grouping wallets by type. Rather than filtering inside JSX, a `groups` array is computed once from the current wallet list. Each element carries a `key`, a `label`, and a `list` of wallets matching that type. System wallets are separated from ordinary wallets so they cannot be accidentally included in the fixed or variable groups. The `groups.map` in the JSX skips any group whose `list` is empty, so no empty section headings appear.

```
const groups = [
  { key: 'fixed',    label: 'Fixed wallets',    list: wallets.filter(w => w.type === 'fixed'
  { key: 'variable', label: 'Variable wallets', list: wallets.filter(w => w.type === 'variable'
  { key: 'system',   label: 'System',           list: wallets.filter(w => w.is_system) },
]
```

Creating and editing through a shared modal. `openCreate` clears `editWallet` to `null` before opening the modal. `openEdit` sets `editWallet` to the chosen wallet. Both then set `modalOpen` to `true`. The `WalletModal` component receives `editWallet` as its `wallet` prop; when that prop is `null`, the modal shows a "New wallet" title and inserts on save. When it is a wallet object, the modal pre-fills and updates on save.

```
async function handleSave(values) {
  if (editWallet) {
    await supabase.from('wallets').update(values).eq('id', editWallet.id)
  } else {
    await supabase.from('wallets').insert(values)
  }
  setModalOpen(false)
  setEditWallet(null)
```

```
  fetchWallets()
}
```

Delete with a system-wallet guard. `handleDelete` checks `wallet.is_system` before proceeding. The system wallet (the Unallocated wallet) must never be deleted; this guard is a safety net in case the UI guard (hiding the delete button on system wallets) is bypassed. A non-system wallet is deleted with a single Supabase call; the cascade on the `wallet_id` foreign key removes all its transactions automatically (see section 3.4.4).

```
async function handleDelete(wallet) {
  if (wallet.is_system) return
  await supabase.from('wallets').delete().eq('id', wallet.id)
  setDeleteTarget(null)
  fetchWallets()
}
```

Imports: `supabase` from `../lib/supabase`, `WalletCard`, `WalletModal`

Exports: `Wallets` (default)

Used by: App.jsx (routed to `/wallets`)

4.1.3.2 pages/WalletDetail.jsx

`WalletDetail.jsx` is the individual wallet page and the most complex file in the project. It reads the wallet's `id` from the URL using `useParams`, fetches the wallet record, its active recurring rules, and all its transactions simultaneously, then renders entirely different content based on `wallet.type`. Fixed wallets show a pending-payments checklist, an upcoming-payments overview, and a recurring-rules manager, all across an Overview and a History tab. Variable wallets show a transaction entry form, a transaction overview table, a full history view, and a spending trends chart across three tabs. Investment wallets show a placeholder. The Unallocated wallet shows a read-only credit transaction list. The file is the assembly hub for all wallet-type-specific components.

Reading the URL and loading data. `useParams` from React Router reads the `:id` segment of the current URL. `fetchAll` uses `Promise.all` to send three Supabase queries in parallel (section 2.7.4), destructuring the results immediately. The recurring-rules query filters for `end_date is null` to exclude retired rules. The `useEffect` dependency is `[id]`, meaning the page reloads whenever the user navigates from one wallet detail to another without unmounting.

```
const { id } = useParams()
const [{ data: w }, { data: r }, { data: t }] = await Promise.all([
  supabase.from('wallets').select('*').eq('id', id).single(),
  supabase.from('recurring_rules').select('*')
```

```
    .eq('wallet_id', id).is('end_date', null).order('created_at'),
  supabase.from('transactions').select('*').eq('wallet_id', id),
])
```

The spending bar calculation. Before any JSX is reached, `WalletDetail` computes the current month's debit total and the corresponding percentage of budget for variable wallets. This computation runs on the already-loaded `transactions` array rather than making a separate query. The date comparison uses ISO strings directly (`t.date >= monthFrom`) which works because date strings in `yyyy-MM-dd` format sort lexicographically. `barColour` and `textColour` apply a traffic-light scheme: green below 75%, amber between 75% and 100%, red at or above 100%.

```
const monthDebits = wallet.type === 'variable'
  ? transactions
      .filter(t => t.type === 'debit' && t.date >= monthFrom && t.date <= monthTo)
      .reduce((s, t) => s + Number(t.amount), 0)
  : 0
const pct       = budget > 0 ? (monthDebits / budget) * 100 : 0
const barColour = pct >= 100 ? 'bg-red-500' : pct >= 75 ? 'bg-amber-400' : 'bg-green-500'
```

The header. The header row is the same for all wallet types. It contains a back-navigation button, the wallet's colour dot and name, and on the right side: the compact spending bar (variable wallets only), the balance pill, and a Settings button (non-system wallets only). The balance pill switches between green and red depending on whether the balance is positive or negative.

```
<div className={`px-4 py-1.5 rounded-full text-sm font-semibold ${
  Number(wallet.balance) >= 0
    ? 'bg-green-50 text-green-700'
    : 'bg-red-50 text-red-600'
}`}>
  Balance: €{Number(wallet.balance).toFixed(2)}
</div>
```

Fixed wallet content. The fixed wallet section uses two tabs: Overview and History. The Overview tab stacks three panels: `TransactionChecklist` (pending payments), `UpcomingPayments` (schedule view), and `RecurringRules` (the rule manager). Each is wrapped in its own white card border. `TransactionChecklist` receives an `onBalanceChanged` callback pointing to `fetchAll`, so confirming a payment re-fetches the wallet's current balance and updates the header pill.

```
{tab === 'overview' && (
  <div className="space-y-6">
    <div className="bg-white rounded-xl border border-gray-200 p-6">
      <TransactionChecklist walletId={id} onBalanceChanged={fetchAll} />
    </div>
    <div className="bg-white rounded-xl border border-gray-200 p-6">
```

```
      <UpcomingPayments rules={rules} transactions={transactions} />
    </div>
    <div className="bg-white rounded-xl border border-gray-200 p-6">
      <RecurringRules walletId={id} onRulesChanged={fetchAll} />
    </div>
  </div>
)}
```

Variable wallet content. The variable wallet section uses three tabs: Overview, History, and Trends. The Overview tab renders `VariableOverview`, which handles transaction entry and the this-week/this-month table. The History tab renders `VariableHistory`, which provides the full sortable, filterable, paginated transaction history. The Trends tab renders `WalletTrendsChart`. Both `VariableOverview` and `VariableHistory` manage their own data fetching; they receive `walletId` as their only data prop.

```
{wallet.type === 'variable' && (
  <>
    {tab === 'overview' && (
      <VariableOverview walletId={id} onBalanceChanged={fetchAll} />
    )}
    {tab === 'history' && (
      <div className="bg-white rounded-xl border border-gray-200 p-6">
        <VariableHistory walletId={id} />
      </div>
    )}
    {tab === 'trends' && <WalletTrendsChart walletId={id} wallet={wallet} />}
  </>
)}
```

The callback pattern. `onBalanceChanged` and `onRulesChanged` are both set to `fetchAll`. When a sub-component changes data that affects the wallet's balance or rule list, it calls its callback using optional chaining (`onBalanceChanged?.()`, see section 2.3.6), which triggers a full re-fetch. The re-fetched wallet data flows back through state into the header, so the balance pill and spending bar reflect the latest values without requiring sub-components to manage that state themselves.

Imports: `supabase` from `../lib/supabase`, `RecurringRules`, `TransactionChecklist`, `UpcomingPayments`, `PaymentHistory`, `WalletModal`, `VariableOverview`, `VariableHistory`, `WalletTrendsChart`

Exports: `WalletDetail` (default)

Used by: App.jsx (routed to `/wallets/:id`)

4.1.3.3 components/WalletCard.jsx

`WalletCard` renders the summary tile for one wallet on the Wallets list page. Clicking anywhere on the card navigates to the wallet's detail page. System wallets show a lock badge instead of edit and delete buttons.

Click navigation and stopPropagation. The outer `div` carries an `onClick` that calls `navigate('/wallets/' + wallet.id)`. The edit and delete buttons each call `e.stopPropagation()` before their own handler, preventing the click event from bubbling up to the div and triggering navigation. This is the standard pattern for action buttons inside a larger clickable area.

```
<button
  onClick={e => { e.stopPropagation(); onEdit(wallet) }}
  className="p-1.5 text-gray-400 hover:text-indigo-600 hover:bg-indigo-50 rounded-lg"
>
  <Pencil size={14} />
</button>
```

System wallet handling. When `wallet.is_system` is true, the edit and delete buttons are replaced by a lock badge. The badge is purely informational: it signals that this wallet is managed by the system and cannot be modified.

Conditional budget and balance display. The card shows a monthly budget figure for fixed and variable wallets but hides it for investment and unallocated wallets. The Unallocated wallet shows its balance instead, since it has no budget. The balance text colour switches between green and red based on sign, consistent with the balance pill in WalletDetail.

Imports: none (project files)

Exports: `WalletCard` (default)

Used by: Wallets.jsx

4.1.3.4 components/WalletModal.jsx

`WalletModal` is the create/edit modal for wallets. The same component handles both paths: when the `wallet` prop is `null` the form is blank and will insert; when it is a wallet object the form is pre-filled and will update. The optional-chaining and null-coalescing pattern (section 2.3.6) initialises every state variable from the wallet prop where it exists, falling back to a sensible default otherwise.

State initialisation from the wallet prop. Each field of the form is its own piece of state, pre-populated at declaration time rather than via a separate `useEffect`. The cap reduction rate is stored

in the database as a fraction between 0 and 1, but displayed in the form as a percentage between 1 and 100. The initialisation converts it: `Math.round(Number(wallet.cap_reduction_rate) * 100)` turns `0.50` into `50` for the input.

```
const [name,       setName]       = useState(wallet?.name        ?? '')
const [type,       setType]       = useState(wallet?.type        ?? 'fixed')
const [budgetType, setBudgetType] = useState(wallet?.budget_type ?? 'fixed-recurring')
const [capReductionRate, setCapReductionRate] = useState(
  wallet?.cap_reduction_rate ? String(Math.round(Number(wallet.cap_reduction_rate) * 100)) : '5
)
```

The BUDGET_TYPES map and the type-change effect. `BUDGET_TYPES` is a module-level object that maps each wallet type to the list of valid budget-type options. When the user changes the `type` selection, a `useEffect` runs to check whether the current `budgetType` is still valid for the new type. If not, it resets to the first available option. This prevents the form from being submitted with an illegal combination such as type `variable` and budget type `fixed-recurring`.

```
useEffect(() => {
  const options = BUDGET_TYPES[type]
  if (!options.find(o => o.value === budgetType)) {
    setBudgetType(options[0].value)
  }
}, [type])
```

Cap reduction settings. A section at the bottom of the form appears only when `type === 'variable'` and `budgetType === 'capped'`. It contains a toggle switch for `cap_reduction_enabled` and, when enabled, a percentage input for `cap_reduction_rate`. The toggle is implemented as a `<button>` whose visual state reflects the boolean via Tailwind classes, a common pattern for custom toggle switches in projects that do not use a UI component library.

Payload construction. `handleSave` builds a single `payload` object and calls `onSave(payload)`. For capped wallets, it adds the cap reduction fields to the payload. The rate is converted from the user-facing percentage back to a fraction: `Number(capReductionRate) / 100`. If reduction is disabled, the rate is stored as `1.0` rather than `0`, so that if the user later re-enables reduction, the previous percentage is remembered in the input but the stored value reflects the disabled state correctly.

```
const payload = { name: name.trim(), type, budget_type: budgetType,
                  budget: Number(budget) || 0, colour, sort_order: Number(sortOrder) || 0 }
if (type === 'variable' && budgetType === 'capped') {
  payload.cap_reduction_enabled = capReductionEnabled
  payload.cap_reduction_rate    = capReductionEnabled ? Number(capReductionRate) / 100 : 1.0
}
await onSave(payload)
```

Imports: none (project files)

Exports: `WalletModal` (default)

Used by: Wallets.jsx, WalletDetail.jsx

4.1.3.5 components/RecurringRules.jsx

`RecurringRules` manages the list of recurring payment rules for a fixed wallet. It renders the list of active rules, provides a create/edit form inline, and handles both soft-deletes and the versioned-edit behaviour described in section 3.5.5. The form UI adapts its visible fields to the selected frequency, showing only the inputs relevant to the schedule type the user has chosen.

The emptyForm constant and setField helper. `emptyForm` is a module-level constant holding blank defaults for every form field. `setField` is a one-line helper that updates one field of the form object using the spread-and-override pattern from section 2.3.3: `setForm(f => ({ ...f, [key]: val }))`. This pattern keeps every input's `onChange` handler short.

```
const emptyForm = {
  name: '', description: '', amount: '',
  frequency: 'monthly', start_date: format(new Date(), 'yyyy-MM-dd'),
  day_of_month: '1', quarter_month: '1', yearly_month: '0',
  custom_dates: [], custom_cycle_years: 1,
}
function setField(key, val) { setForm(f => ({ ...f, [key]: val })) }
```

Frequency-specific UI. The form contains five conditional sections that appear only for their respective frequency. Each is controlled by `{form.frequency === '...' && (...)}`. The weekly section renders a weekday dropdown. The monthly section renders a day-of-month number input. The quarterly section renders a month-within-quarter dropdown and a day input. The yearly section renders a month dropdown and a day input. The custom section renders a full calendar-based date picker sub-component `CustomDatePicker`. This keeps the common fields always visible while showing only what is relevant.

Payload construction. `handleSave` builds a payload with all frequency columns set to `null` first, then populates only the columns relevant to the selected frequency via a `switch` statement. Setting irrelevant columns to `null` explicitly on every save prevents stale values from a previous frequency from persisting in the database row.

```
const payload = { wallet_id: walletId, name: ..., amount: ..., frequency: ...,
  day_of_month: null, quarter_month: null, yearly_month: null,
  custom_dates: null, custom_cycle_years: null, end_date: null, parent_rule_id: null }
```

```
switch (form.frequency) {
  case 'monthly':   payload.day_of_month  = Math.min(Math.max(Number(form.day_of_month), 1), 31
  case 'quarterly': payload.day_of_month  = ...; payload.quarter_month = ...; break
  case 'yearly':    payload.day_of_month  = ...; payload.yearly_month  = ...; break
  case 'custom':    payload.custom_dates  = form.custom_dates; break
}
```

Versioned edit when the amount changes. When editing an existing rule, the component compares the submitted amount to the stored amount. If they differ, it performs an archive-and-replace: it sets `end_date` on the old row to today (retiring it) and inserts a new row with `parent_rule_id` pointing at the retired one. If only non-amount fields changed, a simple `update` is performed instead.

```
if (editingId) {
  const original      = rules.find(r => r.id === editingId)
  const amountChanged = original && Number(original.amount) !== payload.amount
  if (amountChanged) {
    await supabase.from('recurring_rules')
      .update({ end_date: format(new Date(), 'yyyy-MM-dd') }).eq('id', editingId)
    await supabase.from('recurring_rules')
      .insert({ ...payload, parent_rule_id: editingId })
  } else {
    await supabase.from('recurring_rules').update(payload).eq('id', editingId)
  }
}
```

Soft-delete. `handleDelete` sets `end_date` to today on the target rule rather than deleting the row. This is the soft-deletion convention introduced in section 3.2. Past transactions that reference the rule's id remain valid; the rule simply no longer appears in any active-rules query.

```
async function handleDelete(id) {
  await supabase.from('recurring_rules')
    .update({ end_date: format(new Date(), 'yyyy-MM-dd') }).eq('id', id)
  fetchRules()
}
```

Imports: `supabase` from `../lib/supabase`, `formatFrequency` from `../lib/recurringUtils`

Exports: `RecurringRules` (default)

Used by: WalletDetail.jsx

4.1.3.6 components/TransactionChecklist.jsx

`TransactionChecklist` renders the pending-payments checklist for a fixed wallet. It computes which scheduled payments have not yet been confirmed and presents each as a clickable item. Clicking opens a confirmation modal with an optional remark field. Confirming inserts a transaction row (or updates an existing unconfirmed one) and calls `decrement_wallet_balance`. This is precisely the action traced through all layers of the application in section 2.10.

Loading rules and transactions in parallel. Two queries run simultaneously via `Promise.all` (section 2.7.4): one for the wallet's active recurring rules and one for all its transactions. Both results land in state; `buildPendingItems` reads from that state each time the component renders.

```
const [{ data: r }, { data: t }] = await Promise.all([
  supabase.from('recurring_rules').select('*')
    .eq('wallet_id', walletId).is('end_date', null),
  supabase.from('transactions').select('*').eq('wallet_id', walletId),
])
```

Building the pending list. `buildPendingItems` calls `generatePaymentDates` (section 4.1.1.2) for each rule to produce every due date up to today. For each date it searches the loaded transactions for a matching `recurring_rule_id` and date string. If no confirmed transaction is found, the date is pending. The `existingId` field in the result carries the existing transaction's id if an unconfirmed row already exists, or `null` for dates that have no row at all.

```
function buildPendingItems() {
  const today   = startOfDay(new Date())
  const pending = []
  for (const rule of rules) {
    const dueDates = generatePaymentDates(rule, today)
    for (const date of dueDates) {
      const dateStr  = format(date, 'yyyy-MM-dd')
      ...
      if (!existing || !existing.is_confirmed)
        pending.push({ rule, date, dateStr, existingId: existing?.id ?? null })
    }
  }
  return pending.sort((a, b) => a.date - b.date)
}
```

Confirming a payment. `handleConfirm` branches on whether an existing (unconfirmed) transaction row exists. If it does, the existing row is updated with `is_confirmed: true` and the optional remark. If it does not, a new row is inserted. In either case, `decrement_wallet_balance` is called immediately after to adjust the wallet's stored balance. The `onBalanceChanged` callback is called last via optional chaining, triggering `fetchAll` in `WalletDetail` to update the balance pill in the header.

```
await supabase.from('transactions').insert({
  wallet_id:         walletId,
  recurring_rule_id: confirmItem.rule.id,
  amount:            confirmItem.rule.amount,
  type:              'debit', date: confirmItem.dateStr,
  ...
  is_confirmed: true, completed_at: now,
})
await supabase.rpc('decrement_wallet_balance', {
  p_wallet_id: walletId, p_amount: confirmItem.rule.amount,
})
onBalanceChanged?.()
```

Imports: `supabase` from `../lib/supabase`, `generatePaymentDates` from `../lib/recurringUtils`

Exports: `TransactionChecklist` (default)

Used by: WalletDetail.jsx

4.1.3.7 components/UpcomingPayments.jsx

`UpcomingPayments` presents scheduled future payments in either a table view or a calendar view. It receives the already-loaded `rules` and `transactions` arrays as props from `WalletDetail`, so it makes no database calls of its own. The table view shows payments within a user-selected timeframe (this week or this month). The calendar view is a separate sub-component `CalendarView` with its own month-navigation state.

Table view: generating upcoming events. For the table, the component calls `generateUpcomingDates` (section 4.1.1.2) for each rule starting from tomorrow and collects dates up to the selected horizon. As soon as a date exceeds the horizon, the inner loop breaks; the remaining future dates are discarded. The events are sorted by date for chronological display.

```
const tableEvents = []
for (const rule of rules) {
  const upcoming = generateUpcomingDates(rule, addDays(today, 1), 60)
  for (const date of upcoming) {
    if (date > horizon) break
    tableEvents.push({ rule, date, dateStr: format(date, 'yyyy-MM-dd') })
  }
}
tableEvents.sort((a, b) => a.date - b.date)
```

CalendarView: building the byDay map. `CalendarView` combines `generatePaymentDates` (for past and present dates in the viewed month) with `generateUpcomingDates` (for future dates) to find all payment events in the viewed month. The results are grouped into a `byDay` object keyed by day-of-month. Each event carries a `confirmed` flag (from crossing the transactions array) and an `isFuture` flag. Calendar cells with events are colour-coded: blue for future, green for confirmed, red for overdue.

```
const tx        = transactions.find(
  t => t.recurring_rule_id === rule.id && t.date === dateStr
)
const confirmed = tx?.is_confirmed ?? false
const isFuture  = date > today
if (!byDay[d]) byDay[d] = []
byDay[d].push({ rule, date, dateStr, confirmed, isFuture })
```

Calendar grid construction. The grid is a 7-column CSS grid. Empty cells are prepended to align the first day of the month with its correct weekday column. The offset uses the Sunday-to-Monday conversion `(new Date(year, month, 1).getDay() + 6) % 7` to ensure the grid always starts on Monday regardless of the JavaScript `Date` convention.

Imports: `generatePaymentDates`, `generateUpcomingDates` from `../lib/recurringUtils`

Exports: `UpcomingPayments` (default)

Used by: WalletDetail.jsx

4.1.3.8 components/PaymentHistory.jsx

`PaymentHistory` renders the full confirmed-payment history for a fixed wallet in a sortable, filterable, paginated table. Clicking any row opens a detail modal; from the detail modal the user can open an edit form. If the user changes the amount on a historical payment, the component corrects the wallet balance by reversing the old effect and applying the new one.

Relational join in the fetch. The Supabase query uses the dot-notation `select('*, recurring_rules(name, description)')` to fetch the rule's name and description alongside each transaction row. This is a Supabase convenience for PostgreSQL joins: it follows the `recurring_rule_id` foreign key and attaches the referenced row's named columns as a nested object. The result is available as `t.recurring_rules?.name` on each transaction in the component.

```
const { data } = await supabase
  .from('transactions')
  .select('*, recurring_rules(name, description)')
  .eq('wallet_id', walletId)
```

```
  .eq('is_confirmed', true)
  .order('date', { ascending: false })
```

Sort state and the sort pipeline. `sort` holds a `{ key, dir }` object. `toggleSort` either flips the direction if the same column is clicked again, or resets to descending order for a new column. The `sorted` array is produced by copying `filtered` with `[...filtered].sort(...)` rather than sorting in place; sorting mutates an array, and mutating React state directly is incorrect (section 2.4.6).

```
function toggleSort(key) {
  setSort(s =>
    s.key === key
      ? { key, dir: s.dir === 'asc' ? 'desc' : 'asc' }
      : { key, dir: 'desc' }
  )
}
```

Balance correction on edit. When a user submits an edited amount, `submitEdit` does not write to the database directly. Instead it stores a `confirm` object containing the actual write logic as a callback. The confirmation modal calls `confirm.onConfirm()` when the user approves. Inside that callback, if the amount changed, two RPC calls reverse the old balance effect and apply the new one before the transaction row is updated.

```
const amountChanged = Number(f.amount) !== Number(f.oldAmount)
if (amountChanged) {
  await supabase.rpc('increment_wallet_balance',
    { p_wallet_id: walletId, p_amount: Number(f.oldAmount) })
  await supabase.rpc('decrement_wallet_balance',
    { p_wallet_id: walletId, p_amount: Number(f.amount) })
}
```

Imports: `supabase` from `../lib/supabase`

Exports: `PaymentHistory` (default)

Used by: WalletDetail.jsx

4.1.3.9 components/VariableTransactionForm.jsx

`VariableTransactionForm` is the form for logging and editing manual cost entries on a variable wallet. It doubles as a create form (when `editTarget` is `null`) and an edit form (when `editTarget` is a transaction object). Both paths go through a two-step flow: the user fills in the form, clicks the

submit button, and a confirmation modal shows a summary before the database write proceeds.

Pre-population via useEffect. When `editTarget` changes, a `useEffect` (section 2.4.7) fires to populate or reset the form fields. This means the same form instance can be reused for multiple edits without being unmounted: changing `editTarget` from one transaction to another triggers the effect and reloads the form. Using a `useEffect` rather than initialising state from the prop at declaration time is necessary precisely because the prop can change after the component has mounted.

```
useEffect(() => {
  if (editTarget) {
    setName(editTarget.note ?? '')
    setAmount(String(editTarget.amount))
    setDate(editTarget.date)
  } else {
    setName(''); setAmount(''); setDate(todayStr())
  }
}, [editTarget])
```

Two-step confirmation. `handleSubmit` validates the inputs and, if they pass, sets `confirm` to `true`, which renders the modal. The modal shows the transaction summary and calls `handleConfirm` when approved. This separation means validation errors are shown in the form without opening the modal, and the user sees exactly what will be written before it is committed.

The create path and the edit path. For a new transaction, `handleConfirm` inserts a row and calls `decrement_wallet_balance` once. For an edit, it first reverses the old amount with `increment_wallet_balance`, then applies the new amount with `decrement_wallet_balance`, then updates the row. The reversal ensures the balance remains accurate even if the user changes the amount by a large margin.

```
if (editTarget) {
  await supabase.rpc('increment_wallet_balance',
    { p_wallet_id: walletId, p_amount: Number(editTarget.amount) })
  await supabase.rpc('decrement_wallet_balance',
    { p_wallet_id: walletId, p_amount: amt })
  await supabase.from('transactions').update({ amount: amt, date, note: name.trim() })
    .eq('id', editTarget.id)
} else {
  await supabase.from('transactions').insert({ wallet_id: walletId, amount: amt,
    type: 'debit', date, note: name.trim(), is_confirmed: true, completed_at: new Date().toISOS
  await supabase.rpc('decrement_wallet_balance', { p_wallet_id: walletId, p_amount: amt })
}
```

Imports: `supabase` from `../lib/supabase`

Exports: `VariableTransactionForm` (default)

Used by: VariableOverview.jsx

4.1.3.10 components/VariableTransactionList.jsx

`VariableTransactionList` renders the transactions for a variable wallet scoped to a specific calendar month, passed in as the `viewMonth` prop. It is designed to be controlled by a parent that manages which month is in view and when a re-fetch is needed. The `refreshKey` prop is an integer counter: when the parent increments it, `useEffect` re-runs and the list re-fetches without any other prop changing.

Month-scoped fetch. The query uses `.gte('date', from).lte('date', to)` to constrain results to the `viewMonth` boundaries. Both `from` and `to` are formatted as `yyyy-MM-dd` strings. The response is ordered first by date descending, then by `created_at` descending, so same-day entries appear in insertion order within each day.

```
useEffect(() => { fetchTransactions() }, [walletId, viewMonth, refreshKey])

const from = format(startOfMonth(viewMonth), 'yyyy-MM-dd')
const to   = format(endOfMonth(viewMonth),   'yyyy-MM-dd')
const { data } = await supabase.from('transactions').select('*')
  .eq('wallet_id', walletId).gte('date', from).lte('date', to)
  .order('date', { ascending: false }).order('created_at', { ascending: false })
```

Reversing the balance on delete. `handleDelete` checks the transaction's `type` before choosing which balance function to call. A debit that is removed requires an increment (restoring the money). A credit that is removed requires a decrement (undoing the addition). This generalises the balance correction for both directions.

```
if (deleteTarget.type === 'debit') {
  await supabase.rpc('increment_wallet_balance',
    { p_wallet_id: walletId, p_amount: Number(deleteTarget.amount) })
} else {
  await supabase.rpc('decrement_wallet_balance',
    { p_wallet_id: walletId, p_amount: Number(deleteTarget.amount) })
}
await supabase.from('transactions').delete().eq('id', deleteTarget.id)
onChanged?.()
```

Imports: `supabase` from `../lib/supabase`

Exports: `VariableTransactionList` (default)

Used by: not currently wired into the component tree (available for future use in month-navigation views)

4.1.3.11 components/WalletTrendsChart.jsx

`WalletTrendsChart` renders a paired bar chart showing the last six months of spending (debits) and income credits for a variable wallet. The chart is drawn entirely with SVG primitives, consistent with the project's rule of using no external chart libraries. It uses `useMemo` to avoid recomputing the per-month totals on every render.

Fetching six months of transactions. The query fetches only `amount`, `type`, and `date` columns for efficiency, from a computed start date six months ago. The result lands in state and is the sole input to the `useMemo`.

```
const from = format(subMonths(startOfMonth(new Date()), 5), 'yyyy-MM-dd')
const { data } = await supabase.from('transactions').select('amount, type, date')
  .eq('wallet_id', walletId).gte('date', from).order('date', { ascending: true })
```

Computing monthly totals with useMemo. `chartData` is wrapped in `useMemo` (section 2.4.5 covers hooks; `useMemo` caches a computed value and only recomputes when its dependencies change). It builds an array of six month objects, each with a `debit` total, a `credit` total, and a short month label. The `useMemo` dependency is `[transactions]`, so the computation reruns only when the data changes, not on every parent re-render.

```
const chartData = useMemo(() => {
  const months = Array.from({ length: 6 }, (_, i) =>
    subMonths(startOfMonth(new Date()), 5 - i)
  )
  return months.map(m => {
    const mtxns  = transactions.filter(t => t.date >= from && t.date <= to)
    const debit  = mtxns.filter(t => t.type === 'debit') .reduce((s, t) => s + Number(t.amount)
    const credit = mtxns.filter(t => t.type === 'credit').reduce((s, t) => s + Number(t.amount)
    return { month: format(m, 'MMM yy'), debit, credit }
  })
}, [transactions])
```

SVG coordinate helpers. The chart uses a fixed `viewBox` of 560 by 210 pixels with named margin constants (`MT`, `MR`, `MB`, `ML`) for the top, right, bottom, and left margins. The drawing area is the rectangle inside those margins. Three helper functions translate data values to pixel coordinates: `bH` converts a value to a bar height, `bY` converts it to the top y-coordinate of the bar (SVG y increases downward, so taller bars have a smaller y), and `bX` positions the left edge of a bar given its group index and which of the two bars in the pair it is.

```
function bH(val)     { return (val / maxVal) * CH }
function bY(val)     { return MT + CH - bH(val) }
function bX(i, side) {
  const gx = ML + i * slotW + (slotW - groupW) / 2
  return side === 0 ? gx : gx + barW + 3
}
```

Rendering bars and gridlines. The SVG maps over `ticks` to draw horizontal gridlines and y-axis labels, then maps over `chartData` to draw the bar pairs and x-axis month labels. Each bar is a `<rect>` element; the red bar (`fill="#f87171"`) represents spending, the green bar (`fill="#4ade80"`) represents credits. A `<title>` child on each `<rect>` provides a tooltip on hover.

Imports: `supabase` from `../lib/supabase`

Exports: `WalletTrendsChart` (default)

Used by: WalletDetail.jsx

4.1.4 The income feature

Income is the other side of the wallets system. As section 1.2 explains, money enters through one door (the Income page), is divided across wallets according to rules, and any remainder lands automatically in the Unallocated wallet. The distribution logic itself lives in `lib/distributeIncome.js`, already covered in section 4.1.1.3. The three files in this section are the user interface around that logic: the Income page where all three entry workflows live, the recurring income detail page where automated distributions are logged and amended, and the `DistributionPopup` modal that any distribution-requiring action opens.

4.1.4.1 pages/Income.jsx

`Income.jsx` is the most state-heavy page in the application. It manages three distinct income entry workflows from a single tabbed modal: quick entry (a one-off amount with a source name), recurring (a schedule rule with a saved wallet distribution), and template (a saved name/amount pair that can be reused). A single `confirm` state object holds the pending write as a callback, so every destructive action in the page routes through the same confirmation modal. Below the modal system, the page renders the income history table and a two-column grid of recurring income cards and template cards.

Loading data with six parallel queries. `fetchAll` fires once on mount via `useEffect` (section 2.4.7). It sends six queries simultaneously with `Promise.all` (section 2.7.4): income entries, all recurring rules, templates, active wallets, the strict-distribution setting, and the id of the Unallocated wallet. The settings query reads only the `strict_distribution` column via a

`select('strict_distribution')` call, avoiding pulling unneeded data. The Unallocated wallet is fetched separately by filtering `is_system = true`.

```
const [{ data: e }, { data: r }, { data: t }, { data: w }, { data: s }, { data: ua }] = await P
  supabase.from('income_entries').select('*').order('date', { ascending: false }),
  supabase.from('income_recurring').select('*').order('start_date', { ascending: true }),
  supabase.from('income_templates').select('*').order('created_at', { ascending: true }),
  supabase.from('wallets').select('*').eq('is_active', true).order('sort_order'),
  supabase.from('settings').select('strict_distribution').single(),
  supabase.from('wallets').select('id').eq('is_system', true).single(),
])
```

The confirm-as-callback pattern. Every action that writes to the database goes through a two-step confirmation. Rather than having a global `handleConfirm` with a long if/else chain tracking which action is in progress, the page stores the entire write logic inside the `confirm` state object as an `onConfirm` function. When the user presses the confirm button, the modal calls `confirm.onConfirm()`. This means each submit function is self-contained: it packages its own logic into an object and hands it to the modal, which knows only how to display it and call it.

```
setConfirm({
  title: 'Add income?',
  body: <span>Add <strong>{fmt(amount)}</strong> from <strong>{source}</strong>?</span>,
  confirmLabel: 'Add income',
  variant: 'primary',
  onConfirm: async () => {
    await supabase.from('income_entries').insert({ ... })
    setDistributionState({ mode: 'income', totalAmount: Number(amount), ... })
  },
})
```

Quick entry: two post-confirmation paths. `submitQuick` validates the form and builds a `confirm` object. The `onConfirm` callback inside it branches: if this is an edit (`modal.editEntry` is truthy), it calls `update` on the existing row and closes everything. If this is a new entry, it inserts a row and then sets `distributionState` to mode `'income'`, which causes the `DistributionPopup` to appear so the user can split the new money across wallets. The edit path does not open the popup because the money was already distributed at the time of the original entry.

Recurring income: versioned edits and mandatory distribution setup. `submitRecurring` handles both creating a new rule and editing an existing one. The amount-change check is performed before the confirmation: if the submitted amount differs from the stored amount, the confirmation title and body change to warn that the current version will be archived. Inside `onConfirm`, if an amount change was detected, the old row is retired with `end_date` and a new row is inserted with `parent_rule_id` pointing at it, exactly mirroring the versioning pattern used for fixed-wallet recurring rules (section 4.1.3.5). After creating a new rule, `distributionState` is set to mode

'recurringSetup', which opens the popup in its mandatory strict mode where the user must configure the distribution for every future occurrence.

```
if (f.isEdit && amountChanged) {
  await supabase.from('income_recurring').update({ end_date: todayStr() }).eq('id', f.id)
  const { data: newRule } = await supabase.from('income_recurring')
    .insert({ ...payload, start_date: todayStr(), parent_rule_id: f.id }).select().single()
  if (newRule) setDistributionState({ mode: 'recurringSetup',
    ruleId: newRule.id, ruleName: payload.name, ruleAmount: Number(f.amount) })
}
```

Template management and the log-template flow. Templates are simple rows in the `income_templates` table. Creating and editing them is straightforward: `submitTemplate` builds a payload and calls `insert` or `update`. The more interesting path is logging an occurrence: clicking a template card sets `logTemplate` state, which opens a small modal pre-filled with the template's amount but allowing the user to override it for that occurrence. Confirming inserts an `income_entries` row with `source_type: 'template'` and the template's `id` as a foreign key, then sets `distributionState` to mode `'income'` to trigger the distribution popup. Archiving a recurring rule is a single `update` setting `end_date`; templates are the only income object that can be truly hard-deleted.

History filtering and sorting with useMemo. The history table applies a `useMemo` pipeline to the entries array before rendering. The pipeline copies the list to avoid mutating state, applies the `sourceType` filter, applies the search filter using `includes` on the lowercased source name, then sorts by the selected column. The `displayedEntries` slice applies the `histLimit` after filtering, so the row count shown in the "Show N" dropdown reflects the filtered total, not the raw total.

```
const filteredEntries = useMemo(() => {
  let list = [...entries]
  if (histFilter.sourceType !== 'all')
    list = list.filter(e => e.source_type === histFilter.sourceType)
  if (histFilter.search)
    list = list.filter(e => e.source?.toLowerCase().includes(histFilter.search.toLowerCase()))
  list.sort(...)
  return list
}, [entries, histFilter, histSort])
```

The distributionState machine. `distributionState` is either `null` (no popup) or an object with a `mode` field. Mode `'income'` renders `DistributionPopup` with `strictMode` read from the settings and with an `onConfirm` that builds the final distributions array (adding any remainder to Unallocated in non-strict mode) and calls `distributeIncome` with `isAutomated: false`. Mode `'recurringSetup'` renders `DistributionPopup` with `strictMode={true}` and `onClose={null}` (which hides the close button, making distribution setup mandatory), and an `onConfirm` that inserts rows into `income_distribution_rules` rather than executing an immediate distribution.

```
onConfirm={async (distributions) => {
  const finalDists = [...distributions]
  if (!strictMode) {
    const rem = Number((distributionState.totalAmount - assigned).toFixed(2))
    if (rem > 0.005 && unallocatedWalletId)
      finalDists.push({ wallet_id: unallocatedWalletId, amount: rem })
  }
  await distributeIncome({ distributions: finalDists, wallets: allWallets,
    unallocatedWalletId, sourceName: distributionState.sourceName,
    date: distributionState.date, isAutomated: false })
  setDistributionState(null)
}}
```

Imports: supabase from ../lib/supabase, generateUpcomingDates from ../lib/recurringUtils, distributeIncome from ../lib/distributeIncome, DistributionPopup, IncomeConfirmModal

Exports: Income (default)

Used by: App.jsx (routed to /income)

4.1.4.2 pages/IncomeRecurringDetail.jsx

IncomeRecurringDetail.jsx is the detail page for a single recurring income rule, reached by clicking a recurring income card on the Income page. The URL contains the rule's id (section 2.7.4 on useParams), which the page reads to fetch the specific rule, all rules (needed to reconstruct the amendment chain), the rule's distribution setup, and the wallet list. From this page the user can log a new income occurrence (which runs automated distribution if rules are configured), edit the rule's name, frequency, or amount, and view or reconfigure the wallet distribution.

Fetching five parallel queries. fetchData depends on [id] in its useEffect, so it re-runs whenever the URL changes. Five queries run in parallel: the specific rule by id, all income recurring rows (for the chain), the distribution rules for this rule ordered by priority, all active wallets, and the Unallocated wallet's id. The distribution rules are queried with .eq('income_recurring_id', id) so the result is already scoped to this rule.

```
const [{ data: r }, { data: all }, { data: dr }, { data: w }, { data: ua }] = await Promise.all
  supabase.from('income_recurring').select('*').eq('id', id).single(),
  supabase.from('income_recurring').select('*').order('start_date', { ascending: true }),
  supabase.from('income_distribution_rules').select('*').eq('income_recurring_id', id).order('p
  supabase.from('wallets').select('*').eq('is_active', true).order('sort_order'),
  supabase.from('wallets').select('id').eq('is_system', true).single(),
])
```

Reconstructing the amendment chain. `buildChain` is a module-level function that takes a rule id and the full list of all rules and follows `parent_rule_id` links backwards from the given rule to the earliest ancestor. It starts with the current rule, finds its parent in `allRules`, finds that rule's parent, and continues until a rule with no `parent_rule_id` is reached. Because the loop follows links backward (each newer rule points at its predecessor), the chain is built in reverse chronological order and then reversed at the end.

```
function buildChain(ruleId, allRules) {
  const chain = []
  let current = allRules.find(r => r.id === ruleId)
  while (current) {
    chain.push(current)
    if (!current.parent_rule_id) break
    current = allRules.find(r => r.id === current.parent_rule_id)
  }
  return chain.reverse()
}
```

The `chain` is computed via `useMemo` (section 2.4.5) with `[rule, allRules]` as dependencies. This means it is only recomputed when the fetched data changes, not on every render triggered by modal state changes.

The salary growth chart. `SalaryBarChart` is a sub-component defined within the same file. It receives the `chain` array and draws a bar for each rule version. The chart uses the same SVG coordinate pattern as `WalletTrendsChart` (section 4.1.3.11): constants for the viewport size and margins, and helper functions `bH`, `bY`, and `bX` that convert amounts to pixel heights and positions. Active rules (no `end_date`) are drawn in indigo; archived rules are drawn in a lighter shade.

```
const maxAmt = Math.max(...chain.map(r => Number(r.amount)))
const barW   = Math.min(slotW * 0.55, 64)
function bH(val) { return (Number(val) / maxAmt) * CH }
function bY(val) { return MT + CH - bH(val) }
function bX(i)   { return ML + i * slotW + (slotW - barW) / 2 }
```

Logging an income occurrence. `submitLog` validates the form and builds a `confirm` object (same pattern as `Income.jsx`). Inside `onConfirm`, it first inserts an `income_entries` row with `source_type: 'recurring'` and `income_recurring_id` linking back to the rule. Then, if `distributionRules` is non-empty, it calls `distributeIncome` with `isAutomated: true`, passing the saved distribution rules as the `distributions` array. The `isAutomated: true` flag activates the cap reduction logic in `distributeIncome` for any capped wallets (section 4.1.1.3). A three-second `distSuccess` flash confirms the distribution completed.

```
if (distributionRules.length > 0) {
  await distributeIncome({
    distributions: distributionRules.map(dr => ({ wallet_id: dr.wallet_id, amount: Number(dr.am
    wallets: allWallets,
    unallocatedWalletId,
    sourceName: rule.name,
    date: logModal.date,
    isAutomated: true,
  })
}
```

Editing a rule: archive-and-replace vs simple update. `submitEdit` follows the same versioning logic as `submitRecurring` in `Income.jsx` and `handleSave` in `RecurringRules.jsx` (section 4.1.3.5). If the amount changed, the current rule row receives an `end_date` and a new row is inserted referencing it via `parent_rule_id`. After an amount-change edit, the page navigates back to `/income` rather than reloading, because the current URL's id now points at an archived rule. A simple edit (name or frequency only) updates the row in place and calls `fetchData` to reload.

```
if (amountChanged) {
  await supabase.from('income_recurring').update({ end_date: todayStr() }).eq('id', rule.id)
  await supabase.from('income_recurring').insert({ ...payload, start_date: todayStr(), parent_r
  setConfirm(null); setEditModal(null)
  navigate('/income')
} else {
  await supabase.from('income_recurring').update({
    name: payload.name, frequency: payload.frequency, day_of_month: payload.day_of_month,
  }).eq('id', rule.id)
  setConfirm(null); setEditModal(null); fetchData()
}
```

Editing the distribution setup. The distribution panel shows the current rules with wallet colour dots and amounts. Clicking Edit or "Set up distribution" opens `DistributionPopup` with `existingRules` pre-filled from the loaded distribution rules. When the popup confirms, `onConfirm` first deletes all existing `income_distribution_rules` rows for this recurring id, then inserts the new set. This delete-and-replace approach avoids updating individual rows and handles the case where the user removes a wallet from the distribution entirely.

```
await supabase.from('income_distribution_rules').delete().eq('income_recurring_id', id)
if (distributions.length > 0) {
  await supabase.from('income_distribution_rules').insert(
    distributions.map((d, i) => ({ income_recurring_id: id,
      wallet_id: d.wallet_id, amount: d.amount, priority: i }))
  )
```

```
}
```

Imports: `supabase` from `../lib/supabase`, `distributeIncome` from `../lib/distributeIncome`, `DistributionPopup`, `IncomeConfirmModal`

Exports: `IncomeRecurringDetail` (default)

Used by: App.jsx (routed to `/income/recurring/:id`)

4.1.4.3 components/DistributionPopup.jsx

`DistributionPopup` is the modal that appears whenever income needs to be split across wallets. It is used in three contexts: distributing a newly logged manual or template income entry (possibly non-strict, where the remainder routes to Unallocated automatically), and configuring the saved distribution rules for a recurring income source (always strict, where the Distribute button stays disabled until 100% is assigned). The `strictMode` prop and the `onClose` prop together determine which mode the popup operates in.

Loading wallets and seeding amounts from existing rules. `fetchWallets` runs once on mount via `useEffect`. It fetches all active, non-system wallets ordered by `sort_order`. System wallets are excluded with `.eq('is_system', false)` because the Unallocated wallet is handled implicitly by the caller, not shown as a manual target. After loading the wallet list, the function seeds the `amounts` state object from the `existingRules` prop, which carries wallet-to-amount pairs from a previously saved distribution. This pre-filling is how the "Edit" distribution button on `IncomeRecurringDetail` opens the popup with the current values already entered.

```
async function fetchWallets() {
  const { data } = await supabase.from('wallets').select('*')
    .eq('is_active', true).eq('is_system', false).order('sort_order')
  const ws = data ?? []
  setWallets(ws)
  const init = {}
  for (const rule of existingRules) {
    if (Number(rule.amount) > 0) init[rule.wallet_id] = String(rule.amount)
  }
  setAmounts(init)
}
```

Derived display values. `assignedTotal`, `remainder`, `diff`, `canConfirm`, and `totalColour` are all computed directly from the current `amounts` state and the `totalAmount` prop on each render. They are not stored in state of their own because they are always fully derivable from state that already exists; storing them separately would risk them going out of sync (section 2.4.6). `canConfirm`

is the critical gating value: in strict mode it is `false` unless `diff` is less than half a cent, allowing for floating-point arithmetic rounding.

```
const assignedTotal = wallets.reduce((sum, w) => {
  const v = Number(amounts[w.id] || 0)
  return sum + (isNaN(v) ? 0 : v)
}, 0)
const diff        = Math.abs(assignedTotal - totalAmount)
const canConfirm  = strictMode ? diff < 0.005 : true
const totalColour = diff < 0.005 ? 'text-green-600' :
  assignedTotal > totalAmount ? 'text-red-600' : 'text-amber-600'
```

The single-wallet shortcut. `handleWalletClick` implements the shortcut described in ARCHITECTURE.md: clicking the wallet's name (not the input) auto-fills the entire remaining amount into that wallet's input. The condition checks that the wallet's current input is empty (`amounts[wallet.id] || 0 === 0`) and that there is a meaningful remainder to fill (`remainder > 0.005`). Clicking a wallet name when it already has a value, or when the total is already fully assigned, has no effect.

```
function handleWalletClick(wallet) {
  if (Number(amounts[wallet.id] || 0) === 0 && remainder > 0.005) {
    setAmounts(prev => ({ ...prev, [wallet.id]: remainder.toFixed(2) }))
  }
}
```

Building the distributions array and calling the parent. `handleConfirm` collects only the wallets with a positive amount into a clean array of `{wallet_id, amount}` objects. It formats each amount with `Number(Number(amounts[w.id]).toFixed(2))` to prevent floating-point noise from entering the database. The array is passed to the `onConfirm` prop; the popup itself does not write to the database. Whether `onConfirm` calls `distributeIncome` or inserts `income_distribution_rules` rows is entirely the caller's concern, keeping the popup reusable across both use cases.

```
function handleConfirm() {
  const distributions = wallets
    .filter(w => Number(amounts[w.id] || 0) > 0)
    .map(w => ({ wallet_id: w.id, amount: Number(Number(amounts[w.id]).toFixed(2)) }))
  onConfirm(distributions)
}
```

Grouped wallet list and the null close handler. Wallets are grouped into `fixed`, `variable`, and `investment` using three `filter` calls, following the same pattern as `Wallets.jsx` (section 4.1.3.1). Empty groups are skipped in the render. The close button and the Cancel button are both conditionally rendered based on whether `onClose` is not null. Passing `onClose={null}` from

`Income.jsx` when opening the recurring-setup popup removes both buttons, making it impossible to dismiss the popup without completing the distribution setup.

Imports: `supabase` from `../lib/supabase`

Exports: `DistributionPopup` (default)

Used by: Income.jsx, IncomeRecurringDetail.jsx

4.1.5 The dashboard

The dashboard is the application's overview page. It draws from every other part of the system: wallet balances, recurring payment rules, income schedules, distribution rules, and transaction history all flow into it. Most of the computation has been extracted into `lib/dashboardCalcs.js`, already documented in section 4.1.1.4. `Dashboard.jsx` itself is a presentation layer: it loads raw data in one fetch, passes it to the calculation functions from that library, and renders the results across four distinct visual sections. The two SVG chart components it renders, `IncomeSpendingChart` and `CashTrendChart`, are covered in sections 4.1.5.2 and 4.1.5.3 respectively.

4.1.5.1 pages/Dashboard.jsx

`Dashboard.jsx` is the root page of the authenticated application. It sends six queries in parallel, stores the raw results in state, then runs a sequence of `dashboardCalcs` functions synchronously in the component body before returning any JSX. Because all data transformations happen after the loading guard, the rendered output always reflects a consistent snapshot of the loaded state without needing `useMemo` for the computed values.

Loading six tables in parallel. `fetchAll` is defined inline inside the `useEffect` callback and fires once on mount. The six parallel queries bring in every table the dashboard needs: active wallets, income recurring rules, payment recurring rules (active only), all transactions, income distribution rules, and income entries. The transactions and income entries are fetched without date filters because the dashboard's historical charts and month calculations require data from multiple past periods; any date-scoping happens inside the `dashboardCalcs` functions rather than at the query level.

```
const [{ data: w }, { data: ir }, { data: rr }, { data: tx }, { data: dr }, { data: ie }] = awa
  supabase.from('wallets').select('*').eq('is_active', true).order('sort_order'),
  supabase.from('income_recurring').select('*'),
  supabase.from('recurring_rules').select('*').is('end_date', null),
  supabase.from('transactions').select('*'),
  supabase.from('income_distribution_rules').select('*'),
  supabase.from('income_entries').select('*'),
])
```

Post-load computations. After the loading guard renders nothing until data arrives, the component body calls eight `dashboardCalcs` functions in sequence. Each call is a plain assignment; the results are local variables used directly in JSX below. The three-month average is built by mapping `subMonths` offsets over an array of integers, a concise form of `[subMonths(now, 1), subMonths(now, 2), subMonths(now, 3)]`.

```
const cash        = calculateProjectedCash(wallets, incomeRecurring, recurringRules, transactio
const months      = Array.from({ length: 6 }, (_, i) =>
  calculateMonthOutlook(addMonths(now, i), incomeRecurring, recurringRules))
const overdue     = getOverduePayments(recurringRules, transactions)
const overspent   = getOverspentWallets(wallets, transactions, now)
const underfunded = getUnderfundedWallets(wallets, recurringRules, transactions, distributionRu
const metrics     = calculateMonthMetrics(now, transactions, incomeEntries)
const prevMonths  = [1, 2, 3].map(n => subMonths(now, n))
const averages    = calculateMonthlyAverage(prevMonths, transactions, incomeEntries)
```

Projected cash position. The first card renders the result of `calculateProjectedCash` (section 4.1.1.4) as a formula line followed by a coloured badge. The badge switches between green and red based on the sign of `cash.projected`. The underlying formula is: total cash held right now across all wallets, plus income expected in the next 30 days from active recurring income rules, minus upcoming fixed payments not yet confirmed.

```
<div className={`inline-block px-6 py-3 rounded-full text-2xl font-bold ${
  cash.projected >= 0 ? 'bg-green-50 text-green-600' : 'bg-red-50 text-red-500'
}`}>
  €{cash.projected.toFixed(2)}
</div>
```

Six-month outlook strip. The strip below the projected cash card maps the `months` array (six `calculateMonthOutlook` results) into a responsive grid of small cards. Each card shows the calendar month and the projected net for that month: expected income minus expected costs, without consulting any actual transaction history. A `TrendingUp` or `TrendingDown` icon switches based on the sign of `projectedNet`, giving an at-a-glance reading of surplus or deficit months.

Needs attention. The "needs attention" section tests `overdue.length`, `overspent.length`, and `underfunded.length` and renders one alert card per non-empty category. Each card has a left border: red for overdue payments and overspent wallets, orange for underfunded wallets. The `hasAlerts` boolean gates the "all clear" message: when no category has results, the section shows a green checkmark instead of any alert. Overdue items are capped at five visible rows to avoid an unwieldy list.

This month's performance and the MetricCard sub-component. The performance section renders four metric values (income, spending, net, savings rate) using the `MetricCard` subcomponent defined at the bottom of the same file. `MetricCard` receives the current-month value and the three-month average and computes the difference internally. The `lowerIsBetter` prop

inverts the colour convention for the spending metric: a spending figure below the average is shown in green rather than red. The `unit` prop switches the difference label between a euro amount and percentage points for the savings rate.

```
function MetricCard({ label, value, current, avg, lowerIsBetter = false, unit = 'eur' }) {
  const diff   = current - avg
  const better = lowerIsBetter ? diff <= 0 : diff >= 0
  const sign   = diff >= 0 ? '+' : '-'
  const amount = unit === 'pp' ? `${Math.abs(diff).toFixed(1)}pp` : `€${Math.abs(diff).toFixed(
  return (
    <div>
      <p className="text-xs font-medium text-gray-500 mb-1">{label}</p>
      <p className="text-xl font-bold text-gray-800">{value}</p>
      <p className={`text-xs mt-1 ${better ? 'text-green-600' : 'text-red-500'}`}>{sign}{amount
    </div>
  )
}
```

Wallet progress bars. The `progressWallets` array is computed inline in the component body by filtering the loaded wallets to fixed and variable types and computing the current month's confirmed debit total for each. The calculation compares transaction dates against `monthStart` and `monthEnd` using `date-fns` comparison functions rather than string comparisons, because the stored balance on each wallet reflects all-time history and not a monthly slice.

```
const progressWallets = wallets
  .filter(w => w.type === 'fixed' || w.type === 'variable')
  .map(w => {
    const spent = transactions
      .filter(t => t.wallet_id === w.id && t.type === 'debit' && t.is_confirmed
        && !isBefore(new Date(t.date), monthStart) && !isAfter(new Date(t.date), monthEnd))
      .reduce((s, t) => s + Number(t.amount), 0)
    return { ...w, spent }
  })
```

The rendered bar for each wallet applies the same traffic-light colour logic as the spending bar in `WalletDetail.jsx` (section 4.1.3.2): green below 75%, orange between 75% and 100%, red at or above 100%. The bar width is clamped to `Math.min(pct, 100)` so an overspent wallet fills the bar completely rather than overflowing its container.

Over-time trends. The final section assembles the `series` array passed to both chart components. The `viewMode` state toggles between `'monthly'` and `'yearly'`, but the toggle buttons are hidden unless the dataset contains at least a full year of recorded data. The `hasYearOfData` check computes the distance from today to the earliest date in the combined transactions and income entries; `effectiveViewMode` falls back to `'monthly'` when less than 365 days of data exists, regardless of

the stored `viewMode`.

```
const hasYearOfData     = earliestDate !== null && differenceInDays(now, earliestDate) >= 365
const effectiveViewMode = hasYearOfData ? viewMode : 'monthly'
if (effectiveViewMode === 'monthly') {
  const monthsBack = Array.from({ length: 12 }, (_, i) => subMonths(startOfMonth(now), 11 - i))
  series = getHistoricalSeries(monthsBack, transactions, incomeEntries, wallets)
    .map(d => ({ label: format(d.month, 'MMM'), income: d.income, spending: d.spending, totalCa
} else {
  const yearsBack = Array.from({ length: 5 }, (_, i) => subYears(startOfYear(now), 4 - i))
  series = getYearlySeries(yearsBack, transactions, incomeEntries, wallets)
    .map(d => ({ label: format(d.year, 'yyyy'), income: d.income, spending: d.spending, totalCa
}
```

The same `series` array, carrying `{ label, income, spending, totalCash }` per period, is passed to both `<IncomeSpendingChart data={series} />` and `<CashTrendChart data={series} />`. Both charts are stateless: they receive data and return SVG.

Imports: `supabase` from `../lib/supabase`, `calculateProjectedCash`, `calculateMonthOutlook`, `getOverduePayments`, `getOverspentWallets`, `getUnderfundedWallets`, `calculateMonthMetrics`, `calculateMonthlyAverage`, `getHistoricalSeries`, `getYearlySeries` from `../lib/dashboardCalcs`, `IncomeSpendingChart`, `CashTrendChart`

Exports: `Dashboard` (default)

Used by: App.jsx (routed to `/`)

4.1.5.2 components/IncomeSpendingChart.jsx

`IncomeSpendingChart` renders a paired bar chart showing income (green) alongside spending (red) for each period in the `data` array. It is purely a rendering component: it receives data from `Dashboard.jsx`, draws SVG, and returns nothing to its parent. The pattern follows the same structure as `WalletTrendsChart` (section 4.1.3.11) and `SalaryBarChart` in `IncomeRecurringDetail.jsx` (section 4.1.4.2): fixed-size viewport with named margin constants, `niceMax` for clean Y-axis ticks, and small helper functions translating values to pixel coordinates.

SVG constants and axis helpers. The viewport is 720 by 240 pixels. `niceMax` rounds any value up to the nearest power-of-10 multiple, producing clean axis ceiling values like €500 rather than €473. `fmtY` formats axis labels compactly, using the `€1k` shorthand for values of €1,000 or more. Both helpers are module-level functions, not part of any component, so they are computed once at module load time.

```
const W = 720, H = 240
const MT = 15, MR = 15, MB = 30, ML = 60
const CW = W - ML - MR
const CH = H - MT - MB

function bH(val)     { return (val / maxVal) * CH }
function bY(val)     { return MT + CH - bH(val) }
function bX(i, side) {
  const gx = ML + i * slotW + (slotW - groupW) / 2
  return side === 0 ? gx : gx + barW + 3
}
```

Bar geometry. Each period occupies one `slotW` slot across the horizontal axis. Within each slot, the two bars together fill 60% of the slot width (`groupW = slotW * 0.6`). The 3-pixel gap between income and spending bars is baked into `barW`. `bX` places the income bar (side 0) at the left edge of the group and the spending bar (side 1) at the left edge plus one bar width plus the gap.

Rendering. The SVG renders four things: Y-axis gridlines with axis labels (mapped over `ticks`), income bars (only when `d.income > 0`), spending bars (only when `d.spending > 0`), and X-axis period labels. Each `<rect>` carries a `<title>` child for hover tooltips. A final baseline `<line>` draws the zero axis across the full chart width.

Imports: none (project files)

Exports: `IncomeSpendingChart` (default)

Used by: Dashboard.jsx

4.1.5.3 components/CashTrendChart.jsx

`CashTrendChart` renders the total cash-over-time line chart. Unlike the bar charts, it uses a connected polyline path built from SVG `M` (move-to) and `L` (line-to) path commands, with a shaded fill beneath the line. It is the only chart in the project that accounts for negative values: `minVal` can be below zero when the combined wallet balance was negative at some past point.

Supporting negative values. The Y-axis range is computed from `minVal = Math.min(0, ...values)` and `maxVal = niceMax(Math.max(...values, 0))`. `minVal` is always at most zero, so the zero line always appears within the chart area. The `py` function maps any value (including negative ones) to a pixel Y coordinate by measuring its position within `range`, the distance from `minVal` to `maxVal`.

```
const values = data.map(d => d.totalCash)
const minVal = Math.min(0, ...values)
```

```
const maxVal = niceMax(Math.max(...values, 0))
const range  = maxVal - minVal || 1
function px(i)   { return ML + i * stepX }
function py(val) { return MT + CH - ((val - minVal) / range) * CH }
```

SVG path construction. `linePath` is built by mapping each data point to a `M x y` command for the first point and `L x y` for all subsequent ones, then joining them into a single path string. `areaPath` extends `linePath` with two additional `L` commands that drop down to the baseline and close the shape with `Z`, creating the filled area beneath the line. Both paths are passed to `<path>` elements: the area fill uses low opacity indigo, the line uses solid indigo at 2-pixel stroke width.

```
const linePath = points.map((p, i) => `${i === 0 ? 'M' : 'L'} ${p.x} ${p.y}`).join(' ')
const areaPath = points.length > 0
  ? `${linePath} L ${points[points.length - 1].x} ${MT + CH} L ${points[0].x} ${MT + CH} Z`
  : ''
```

Rendering. The chart renders gridlines and Y-axis labels (one of which coincides with the zero baseline), then the area fill, then the line, then a dot at each data point. The most recent point's dot is drawn at radius 4 instead of 2.5, visually emphasising the current position. A `<title>` child on each dot provides a hover tooltip with the period label and exact cash figure.

Imports: none (project files)

Exports: `CashTrendChart` (default)

Used by: Dashboard.jsx

4.1.6 Settings

The Settings page manages four application configuration values: the currency display (read-only), the month start day, the theme preference, and the strict-distribution toggle. It also contains a Danger Zone with a "delete all data" workflow for resetting the application's financial history while preserving the wallet structure and rules. Each setting is rendered in a reusable `SettingCard` wrapper component defined in the same file, and the two toggle controls use a shared `Toggle` button component also defined there.

4.1.6.1 pages/Settings.jsx

`Settings.jsx` loads the single row from the `settings` table (see section 3.9), renders the four configuration cards, and updates individual fields on user interaction without requiring a Save button. A "Saved" indicator briefly appears after each change. The delete-all-data workflow is

the most structurally complex part of the file: it opens a two-step modal chain protected by email one-time password verification.

State and two separate fetches. The page runs two `useEffect` fetches on mount: `fetchSettings` reads the settings row via `.single()`, and `fetchUserEmail` calls `supabase.auth.getUser()` to retrieve the logged-in user's email address. The email is needed later as the OTP target. The two fetches are independent and could be run in parallel; the current implementation runs them sequentially from separate function calls, which works correctly because the page's visible content depends only on `settings`, and `userEmail` is not needed until the user opens the delete modal.

Optimistic update with debounced flash. `updateSetting` writes the new value to the database and immediately updates the local `settings` state with the spread-and-override pattern (section 2.3.3), so the UI reflects the change without waiting for a database round trip. A "Saved" indicator appears for two seconds after each write. The cleanup logic uses `useRef` to store the current timeout id: if the user makes another change before the two seconds expire, `clearTimeout` cancels the previous timer and `setTimeout` starts a fresh one.

```
async function updateSetting(field, value) {
  if (!settings?.id) return
  await supabase.from('settings').update({ [field]: value }).eq('id', settings.id)
  setSettings(prev => ({ ...prev, [field]: value }))
  if (timerRef.current) clearTimeout(timerRef.current)
  setSaved(true)
  timerRef.current = setTimeout(() => setSaved(false), 2000)
}
```

`useRef` is being used here as an escape hatch from React's state model: unlike `useState`, updating a ref does not trigger a re-render. Storing the timeout id in a ref means the cleanup can run without causing the component to re-render unnecessarily.

The SettingCard and Toggle sub-components. `SettingCard` is a layout wrapper that places a label and description on the left and its `children` on the right. Every setting row uses it. `Toggle` is a custom toggle-switch button whose visual state reflects the boolean `checked` prop via a Tailwind `translate-x` class on the inner circle. Both are defined at the top of the same file, outside the `Settings` function. The pattern is the same as `MetricCard` in `Dashboard.jsx`: a sub-component written in the same file because it is only used in that one context.

The four setting cards. Currency is displayed as a read-only badge; the value is hardcoded to `EUR €` and the field is not interactive, as multi-currency support is not part of the current scope.

The month start day input clamps its value between 1 and 28 on every change. The upper limit is 28 rather than 31 because months shorter than 31 days would make a start day of 29, 30, or 31 invalid in some months.

The theme toggle persists the `'light'` or `'dark'` string to the `settings` table, but the application does not read this value at startup to apply CSS changes. As the `SettingCard` description notes:

dark mode is saved but not yet applied visually. The database column stores the user's preference for future use when the feature is implemented.

The strict distribution toggle controls how `DistributionPopup` behaves for manual and template income (section 4.1.4.3). When off, unassigned income flows automatically to the Unallocated wallet; when on, the popup's Distribute button stays disabled until 100% is allocated.

The delete-all-data flow: warning modal and OTP modal. The `deleteModal` state is a simple string state machine with three values: `null` (no modal), `'warning'` (first confirmation), and `'code'` (OTP entry). Clicking "Delete all data" sets it to `'warning'`. Clicking "Send confirmation code" in the warning modal calls `sendOtp`, which triggers Supabase's email OTP system and advances the state to `'code'`.

```
async function sendOtp() {
  if (!userEmail) return
  setDeleteLoading(true)
  await supabase.auth.signInWithOtp({
    email: userEmail,
    options: { shouldCreateUser: false },
  })
  setDeleteLoading(false)
  setDeleteModal('code')
}
```

The `shouldCreateUser: false` option prevents Supabase from creating a new account if the email is not found. The OTP flow relies on Supabase's transactional email delivery being configured in the project's dashboard; if email sending is not configured or is rate-limited, the code modal will appear but no email will arrive, and the user will be unable to proceed. This external dependency is worth noting as a potential point of failure in development or staging environments.

confirmDeletion: verifying the OTP and wiping the data. Once the user enters the six-digit code, `confirmDeletion` calls `supabase.auth.verifyOtp`. If verification fails, an error message is shown and nothing is deleted. If it succeeds, four sequential database operations run: deleting all transactions, all income entries, all budget allocations, and resetting all wallet balances to zero. The last operation uses `update` rather than `delete` to preserve the wallet rows themselves.

```
const { error } = await supabase.auth.verifyOtp({ email: userEmail, token: otpCode.trim(), type
if (error) { setDeleteError('Invalid or expired code.'); setDeleteLoading(false); return }
await supabase.from('transactions').delete().neq('id', '00000000-0000-0000-0000-000000000000')
await supabase.from('income_entries').delete().neq('id', '00000000-0000-0000-0000-000000000000'
await supabase.from('budget_allocations').delete().neq('id', '00000000-0000-0000-0000-00000000
await supabase.from('wallets').update({ balance: 0 }).neq('id', '00000000-0000-0000-0000-000000
```

The `.neq('id', '00000000-0000-0000-0000-000000000000')` filter is a known workaround pattern in Supabase applications. When Row Level Security is enabled on a table, Supabase requires

that every `DELETE` and `UPDATE` call include at least one filter clause; a bare `.delete()` is rejected even if the RLS policy permits it. Filtering on `id` not equal to the nil UUID (a string of all zeros that will never match a real auto-generated UUID) satisfies the filter requirement while effectively targeting every row in the table. The operations run one after another rather than in parallel because each deletion must complete before the next to avoid foreign key constraint violations between, for example, transactions and income entries.

After the data wipe, `deleteSuccess` is set to `true`, which renders a toast at the bottom of the screen, and `navigate('/')` fires after 1.5 seconds to return the user to the now-empty dashboard.

Imports: `supabase` from `../lib/supabase`

Exports: `Settings` (default)

Used by: App.jsx (routed to `/settings`)

4.2 The project root

Everything covered in section 4.1 lives inside the `src/` folder. The files sitting directly in the project root configure the machinery around that code: how the app is built, which packages it depends on, how the linter checks it, what the browser receives as its first document, and which files Git should never see. These files are rarely edited after initial setup, but reading them completes the picture of how the project works as a whole.

4.2.1 package.json

`package.json` is the central manifest of a Node.js project. It serves three purposes at once: it declares the project's identity and metadata, it lists every package the project depends on, and it defines the shorthand commands used during development and deployment. Section 2.6.1 explains that `npm install` reads this file and downloads the listed packages into `node_modules`.

Project metadata. Three top-level fields describe the project itself. `"name": "financieel"` is the project identifier. `"private": true` prevents the project from being accidentally published to the public npm package registry. `"type": "module"` tells Node.js that the project's JavaScript files use ES module syntax (`import`/`export`) rather than the older CommonJS syntax (`require`). This matters primarily for config files like `vite.config.js` and `eslint.config.js`, which run in Node.js rather than in a browser.

The scripts block. The four entries in `"scripts"` are the commands run with `npm run <name>`. Section 2.6.2 covers `dev` and section 2.6.3 covers `build`. `preview` serves the production build locally for testing before deploying. `lint` runs ESLint across the whole project.

```
"scripts": {
  "dev":     "vite",
  "build":   "vite build",
  "lint":    "eslint .",
  "preview": "vite preview"
},
```

Runtime dependencies. The `"dependencies"` block lists packages that are bundled into the production build and sent to users' browsers. Each version string begins with `^`, meaning npm accepts any version that is compatible with the listed one (same major version, any higher minor or patch). The seven packages here cover every external capability the application relies on at runtime.

```
"dependencies": {
  "@supabase/supabase-js": "^2.106.0",
  "date-fns":              "^4.2.1",
  "lucide-react":          "^1.16.0",
  "react":                 "^19.2.6",
  "react-dom":             "^19.2.6",
  "react-is":              "^19.2.7",
  "react-router-dom":      "^7.15.1"
},
```

`@supabase/supabase-js` is the client library wrapping all database and authentication requests (section 2.7.3). `date-fns` provides the calendar arithmetic functions used throughout the codebase (section 2.3.4 touches on date handling). `lucide-react` is the icon library. `react` and `react-dom` are React itself; `react-dom` is the package that knows how to translate React's virtual descriptions into real DOM operations. `react-is` is a peer dependency required by certain React internals. `react-router-dom` provides the routing layer (section 2.4.8 and the App.jsx discussion in section 4.1.2.3).

Development dependencies. The `"devDependencies"` block lists packages used only during development and building; they are never included in the production bundle shipped to users. The list includes Vite itself, the Vite plugins for React and Tailwind, the ESLint toolchain, and TypeScript type definitions for React. The type definitions (`@types/react`, `@types/react-dom`) provide IDE autocompletion even though this project is written in plain JavaScript without TypeScript.

```
"devDependencies": {
  "@eslint/js":                "^10.0.1",
  "@tailwindcss/vite":         "^4.3.0",
  "@types/react":              "^19.2.14",
  "@types/react-dom":          "^19.2.3",
  "@vitejs/plugin-react":      "^6.0.1",
  ...
  "tailwindcss":               "^4.3.0",
```

```
  "vite":                         "^8.0.12"
},
```

4.2.2 vite.config.js

`vite.config.js` is Vite's configuration file. It is a JavaScript module that exports a configuration object via Vite's `defineConfig` helper. Section 2.6 explains Vite's role: it translates JSX, bundles the source files and packages, and serves the development server. The config file is where plugins that extend that behaviour are registered.

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
  ],
  ...
  optimizeDeps: {
    include: ['recharts', 'react-is'],
  },
})
```

The plugins array. Two plugins are registered. `react()` from `@vitejs/plugin-react` does two things: it transforms JSX syntax into JavaScript that browsers can run, and it enables React Fast Refresh, the mechanism that pushes component changes into the running browser without a full page reload during development. `tailwindcss()` from `@tailwindcss/vite` integrates Tailwind CSS v4 directly into Vite's pipeline, scanning source files for class names and generating the stylesheet on demand. Section 2.5.3 explains how this scanning works.

The optimizeDeps entry. `optimizeDeps.include` tells Vite to pre-bundle the listed packages when the development server starts, rather than transforming them on first request. `react-is` is a valid runtime dependency listed in `package.json`. `recharts` appears here as a leftover from an earlier version of the project: ARCHITECTURE.md notes that recharts was removed due to a Vite 8 and React 19 incompatibility. The entry has no effect since recharts is no longer installed, but it causes no harm either.

4.2.3 Tailwind CSS configuration

Unlike the previous two major versions of Tailwind (v2 and v3), Tailwind CSS v4 does not use a separate `tailwind.config.js` file. Readers coming from older tutorials or other projects may look for such a file; it does not exist in this project, and that is by design.

In Tailwind v4, configuration is handled through two mechanisms that are already present in the project. The first is the `@tailwindcss/vite` plugin registered in `vite.config.js` (section 4.2.2), which handles content scanning automatically: it reads the same file graph that Vite builds for bundling and knows which source files to scan for class names without needing an explicit `content` array. The second is the single `@import "tailwindcss"` directive in `src/index.css` (section 4.1.2.2), which activates the Tailwind layer system and makes all utility classes available.

Any project-level customisation in v4, such as adding custom colours or defining design tokens, is done with CSS custom property syntax directly in `index.css` rather than in a JavaScript config object. This project uses no custom configuration beyond what Tailwind provides out of the box, so `index.css` remains a single import line.

4.2.4 eslint.config.js

`eslint.config.js` configures ESLint, the static analysis tool that reads source files and flags potential bugs and style problems without running the code. Running `npm run lint` invokes it across the whole project; it also integrates with most code editors to show warnings inline while writing. The file uses ESLint's flat config format, introduced in ESLint v9, which represents the configuration as an array of config objects applied in sequence rather than a nested JSON object.

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [ js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite ],
    languageOptions: {
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true } },
    },
```

```
  },
])
```

globalIgnores. `globalIgnores(['dist'])` tells ESLint to skip the `dist/` folder, which contains the production build output. Linting generated files would produce noise without value.

The files pattern. The `files: ['**/*.{js,jsx}']` pattern scopes the following rules to JavaScript and JSX files only, leaving other file types untouched.

The three rule sets. `js.configs.recommended` applies ESLint's standard JavaScript rules: detecting unused variables, unreachable code, and similar common mistakes. `reactHooks.configs.flat.recommende` enforces the rules of hooks (section 2.4.5): hooks must be called at the top level of a component function, never inside conditions or loops. `reactRefresh.configs.vite` checks that exports from component files are compatible with React Fast Refresh; components that cannot be hot-reloaded would otherwise cause a full page reload on every edit.

Language options. `globals.browser` makes browser-global names (`window`, `document`, `navigator`, `fetch`) known to the linter, preventing false "variable not defined" errors for these built-in values. `ecmaFeatures: { jsx: true }` enables the JSX parser extension so the linter can read files containing JSX syntax.

4.2.5 index.html

`index.html` is the only HTML file in the project and the only file Vite serves directly without transformation. Every request to the application, regardless of the URL path, returns this one document; React Router then reads the URL client-side and renders the matching page component. Section 2.2.1 explains why our project's HTML body is almost empty.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    ...
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

The head metadata. `charset="UTF-8"` instructs the browser to interpret the document using

the UTF-8 character encoding, which supports all international characters. The `favicon.svg` link provides the browser tab icon as a scalable vector file; a single SVG covers all sizes without needing multiple image files. The `viewport` meta tag sets the initial zoom level to match the device's physical width and prevents mobile browsers from zooming out to display a desktop-sized page.

The body. Two elements constitute the entire body. `<div id="root"></div>` is the mount point: `main.jsx` (section 4.1.2.1) targets this element with `document.getElementById('root')` and hands it to React's `createRoot`. Everything the user sees is inserted here by React at runtime; the div itself is empty when the HTML is first sent.

`<script type="module" src="/src/main.jsx">` loads the application. The `type="module"` attribute tells the browser to treat the script as an ES module, enabling `import` syntax in the browser. In development, Vite intercepts this request, transforms the JSX file, and returns the result. In production, the `build` step replaces this reference with the path to the compiled and hashed bundle file; the HTML file itself is rewritten as part of the build output.

4.2.6 .gitignore

`.gitignore` is a plain text file read by Git when deciding which files to include in the repository. Each line is a pattern; files and folders matching any pattern are excluded from all Git operations: they are not tracked, not staged, and not committed. The file has no effect on the running application; it exists purely to keep the repository clean and, critically, to keep secrets out.

The file contains patterns in four groups. Log files (`*.log` and several named variants) are excluded because they contain runtime noise rather than source code. `node_modules`, `dist`, and `dist-ssr` are excluded because they are outputs rather than inputs: `node_modules` is the downloaded package directory (section 2.6.1), and `dist` and `dist-ssr` are Vite's production build outputs. All three can be regenerated from the source files and `package.json`.

```
node_modules
dist
dist-ssr
*.local
```

The `*.local` pattern is the most security-critical line in the file. It matches any file whose name ends in `.local`, which includes `.env.local`, the file holding the Supabase project URL and the anon key (section 2.6.3). If `.env.local` were committed to the repository, its contents would be visible to anyone with read access to the repository on GitHub and would persist in the commit history permanently even if later deleted. Section 2.9 explains what protections remain if the anon key is exposed, but the `.gitignore` entry ensures the question never arises in practice.

The final group covers editor-specific directories and temporary files: `.vscode/*` (with an exception for the shared `.vscode/extensions.json` file, which contains extension recommendations useful to collaborators), `.idea` (JetBrains IDEs), `.DS_Store` (macOS folder metadata), and several Visual

Studio and Vim temporary file extensions. These vary by developer and should not become part of the shared project history.